5

Modelagem de classes de análise

> O engenheiro de software amador está sempre à procura da mágica, de algum método sensacional ou ferramenta cuja aplicação promete tornar trivial o desenvolvimento de software. É uma característica do engenheiro de software profissional saber que tal panacéia não existe.
>
> – GRADY BOOCH

O modelo de casos de uso de um sistema é construído para formar a *visão de casos de uso* do sistema (conforme mencionado na Seção 1.4.1). Esta visão fornece uma perspectiva do sistema a partir de um ponto de vista *externo*. De posse da visão de casos de uso, os desenvolvedores precisam prosseguir no desenvolvimento do sistema.

A funcionalidade externa de um sistema orientado a objetos é fornecida por meio de colaborações entre objetos. Externamente ao sistema, os atores visualizam resultados de cálculos, relatórios produzidos, confirmações de requisições realizadas etc. Internamente, os objetos do sistema colaboram uns com os outros para produzir os resultados visíveis de fora. Essa colaboração pode ser vista sob o *aspecto dinâmico* e sob o *aspecto estrutural estático*.

O *aspecto dinâmico* de uma colaboração entre objetos descreve a troca de mensagens entre os mesmos e a sua reação a eventos que ocorrem no sistema. O aspecto dinâmico de uma colaboração é representado pelo Modelo de Interações e é estudado nos Capítulos 7 e 10.

O *aspecto estrutural estático* de uma colaboração permite compreender como o sistema está estruturado internamente para que as funcionalidades externamente visíveis sejam produzidas. Esse aspecto é dito *estático* porque não apresenta informações sobre como os objetos do sistema interagem no decorrer do tempo (isso é representado no aspecto dinâmico da colaboração). Também é

dito *estrutural* porque a estrutura das classes de objetos componentes do sistema e as relações entre elas são representadas.

Os aspectos estáticos e dinâmicos de um sistema orientado a objetos não são independentes. Na verdade, conforme descrito com mais detalhes neste capítulo, a construção de um serve para adicionar detalhes no outro. Por exemplo, quando, durante a construção do aspecto dinâmico, o modelador detecta a necessidade de uma mensagem entre dois objetos, isso implica a existência de alguma referência estrutural entre os mesmos. Do mesmo modo, a definição da estrutura do relacionamento entre dois objetos feita no aspecto estrutural influencia na forma pela qual esses mesmos objetos podem trocar mensagens.

Este capítulo inicia a descrição do aspecto estrutural estático de um sistema orientado a objetos. Esse aspecto é representado pelo Modelo de Classes, da mesma forma que o aspecto funcional é representado pelo Modelo de Casos de Uso. A ferramenta da UML utilizada para representar o aspecto estrutural estático é o *diagrama de classes*. O *modelo de classes* é composto desse diagrama e da descrição textual associada ao mesmo. Para melhor orientar o leitor, os objetivos principais deste capítulo são enumerados a seguir.

1. Descrever algumas técnicas para identificação de classes de análise.
2. Apresentar alguns dos elementos do diagrama de classes necessários à construção do modelo de classes de análise (outros elementos são descritos em capítulos posteriores). É também objetivo deste capítulo apresentar o *diagrama de objetos*, também relacionado ao aspecto estrutural estático.
3. Descrever como o modelo de classes pode ser documentado.
4. Descrever a inserção do modelo de classes de análise em um processo de desenvolvimento iterativo.

5.1 Estágios do modelo de classes

É importante notar que o modelo de classes é utilizado durante a maior parte do desenvolvimento iterativo de um SSOO. Mais que isso, esse modelo evolui durante as iterações do desenvolvimento do sistema. À medida que o sistema é desenvolvido, o modelo de classes é incrementado com novos detalhes. Há três *estágios sucessivos de abstração* pelos quais o modelo de classes passa: *análise*, *especificação* e *implementação*.

1. O *modelo de classes de análise* representa as classes de análise, ou seja, as que se tornam evidentes na medida em que focamos a atenção sobre "o que" o sistema deve fazer. Este modelo é construído na fase de análise (ver Seção 2.1.2). Por definição, um modelo de classes de análise não leva em consideração restrições inerentes à tecnologia a ser utilizada na

solução de um problema. Em seu conjunto, o modelo de casos de uso e o modelo de classes de análise são os dois principais modelos criados na fase de análise (ver Seção 2.1.2).

2. *O modelo de classes de especificação* é um detalhamento do modelo de classes de análise. É também conhecido como *modelo de classes de projeto*. Quando chegamos nesse estágio, normalmente descobrimos a necessidade de criar outras classes, pois começamos a focar nossa atenção sobre "como" o sistema deve funcionar. Além disso, esse detalhamento do modelo de classes também envolve a adição de detalhes às classes identificadas na análise, em função da solução de software escolhida. O modelo de classes de projeto é construído na atividade de projeto (ver Seção 2.1.3) do desenvolvimento de uma iteração do desenvolvimento. Para entender a diferença entre o modelo de classes de análise e o de projeto, vale aqui uma analogia com a construção de uma casa. No modelo de análise de uma casa, pensamos em objetos como salas, quartos, banheiro, portas etc. No modelo de especificação desta mesma casa, temos que pensar em outros detalhes, como encanamento, parte elétrica, caixonetes para as portas, ou seja, detalhes que não são evidentes para os ocupantes de uma casa, mas que são necessários para construí-la propriamente.
3. O *modelo de classes de implementação* é um detalhamento do modelo de especificação. Esse modelo corresponde à *implementação* das classes em alguma linguagem de programação, normalmente uma linguagem orientada a objetos (C++, Java, C# etc.). O modelo de implementação é construído na atividade de implementação (ver Seção 2.1.4) de um processo de desenvolvimento iterativo. (Note que aqui estamos considerando o próprio código-fonte do sistema como um modelo.)

Neste capítulo, nosso principal objetivo é estudar o modelo de classes em seu primeiro estágio. O modelo de classes de análise é composto dos objetos identificados na análise do domínio e na análise da aplicação (ver Seção 2.1.2). Seu objetivo é descrever o problema representado pelo sistema a ser desenvolvido; ele não considera características da solução a ser utilizada. Já os modelos de especificação e de implementação consideram detalhes da solução de software a ser utilizada. No entanto, ao contrário do modelo de implementação, o de especificação descreve a solução em um nível alto de abstração.

Antes de passarmos para as próximas seções, apresentamos a seguir a nomenclatura de identificadores dos elementos do modelo de classes utilizada neste livro. Entretanto, é importante notar que essa nomenclatura é apenas uma possibilidade. A equipe de desenvolvimento pode escolher qualquer estilo de nomenclatura que desejar. O importante é que, uma vez escolhida uma nomenclatura, esta seja utilizada consistentemente.

1. Para identificadores, quaisquer espaços em branco e preposições do nome são removidos.
2. Para nomes de classes e nomes de relacionamentos, as palavras componentes do nome são escritas começando por letra maiúscula. Exemplos: `Cliente`, `ItemPedido`, `Pedido`, `OrdemServiço`, `Reside`, `Realiza` etc.
3. Para nomes de atributos e nomes de operações: deve-se escrever a primeira palavra do nome do atributo em minúsculas. Escreva as palavras subseqüentes em maiúsculas. No entanto, siglas são mantidas inalteradas. Exemplos: `quantidade`, `precoUnitário`, `CPF`, `nome`, `dataNascimento`, `obterTotal` etc.

5.2 Diagrama de classes

O *diagrama de classes* é utilizado na construção do modelo de classes desde o nível de análise até o nível de especificação. De todos os diagramas da UML, esse é o mais rico em termos de notação. Nesta seção, são apresentados os elementos do diagrama de classes utilizados para a construção do modelo de classes de nível de análise.

5.2.1 Classes

Uma classe é representada por uma "caixa" com, no máximo, três compartimentos exibidos. No primeiro compartimento (de cima para baixo) é exibido o nome da classe. Por convenção, esse nome é apresentado no singular e com as palavras componentes começando por maiúsculas. No segundo compartimento, são declarados os atributos, que correspondem às informações que um objeto armazena. Finalmente, no terceiro compartimento, são declaradas as operações, que correspondem às ações que um objeto sabe realizar.

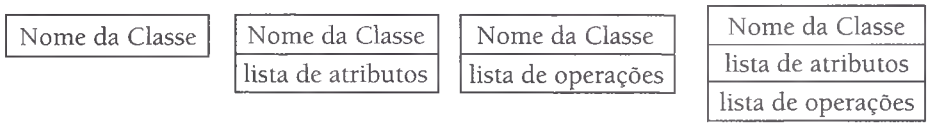


Figura 5-1: Possíveis notações para uma classe na UML.

As possíveis notações da UML para representar classes são apresentadas na Figura 5-1. O grau de abstração desejado em um dado momento do desenvolvimento do modelo de classes direciona a utilização de uma ou outra notação.

Estruturalmente, uma classe é composta de *atributos* e de *operações*. Os atributos correspondem à descrição dos dados armazenados pelos objetos de uma classe. A cada atributo de uma classe está associado um conjunto de valores que esse atributo pode assumir. As operações correspondem à descrição das ações que os objetos de uma classe sabem realizar. Ao contrário dos atributos (para os quais cada ob-

jeto tem o seu próprio valor), objetos de uma classe compartilham as mesmas operações. O nome de uma operação normalmente contém um verbo e um complemento, e terminam com um par de parênteses. (Na descrição do modelo de classes de especificação, é apresentada a verdadeira utilidade desse par de parênteses.)

A Figura 5-2 ilustra exemplos de representação de uma mesma classe, `ContaBancária`, em diferentes graus de abstração.

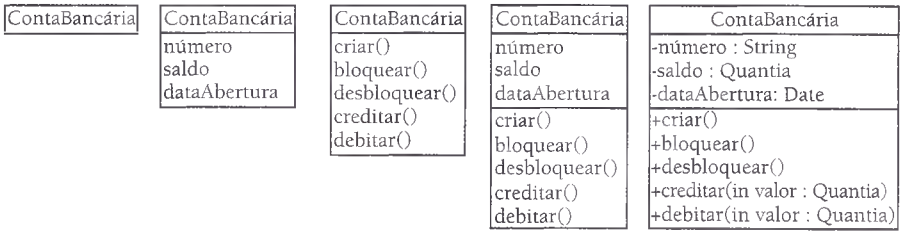


Figura 5-2: Diferentes graus de abstração na notação de classe.

5.2.2 Associações

Da Seção 1.2.1, sabe-se que cada ocorrência de uma classe é chamada de *objeto* ou *instância*. Um ponto importante acerca de objetos de um sistema é o fato de que eles podem se relacionar uns com os outros. A existência de um relacionamento entre dois objetos possibilita a troca de mensagens (ver Seção 7.5.1) entre os mesmos. Portanto, em última análise, relacionamentos entre objetos permitem que eles colaborem entre si a fim de produzir as funcionalidades do sistema.

No diagrama de classes, podemos representar a existência de relacionamentos entre objetos. Para representar o fato de que objetos podem se relacionar uns com os outros, existe outro elemento na notação do diagrama de classes, a *associação*. Esse elemento representa relacionamentos que são formados entre objetos durante a execução do sistema. Uma associação é representada no diagrama de classes por uma linha (normalmente um segmento de reta) ligando as classes às quais pertencem os objetos relacionados. Na Figura 5-3, apresentamos alguns exemplos de associações entre objetos: (1) no domínio de vendas, um *cliente* compra *produtos*; (2) no domínio bancário, uma *conta-corrente* possui um *histórico de transações*; (3) em um hotel, há vários *hóspedes*, assim como há vários *quartos*. Os *hóspedes* do hotel ocupam *quartos*.

É importante notar que, embora as associações sejam representadas entre classes do diagrama, tais associações representam ligações possíveis entre *objetos* das classes envolvidas na associação. Por exemplo, quando ligamos no desenho acima as classes `Hóspede` e `Quarto`, isso significa que, durante a execução do sistema, haverá a possibilidade de troca de mensagens entre objetos dessas classes. Ou seja, objetos dessas classes poderão colaborar para realizar alguma (parte de uma) tarefa do sistema.

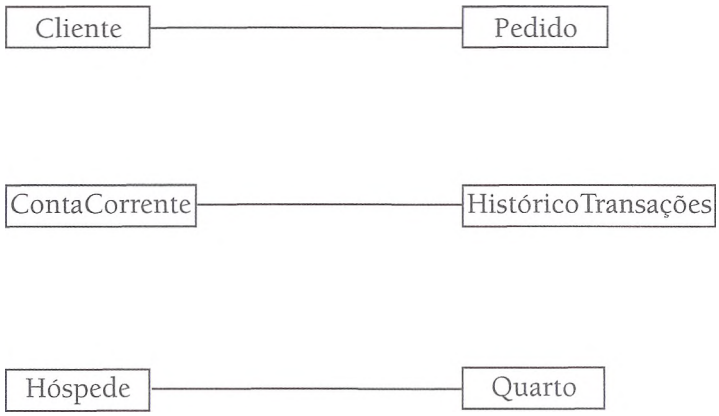


Figura 5-3: Exemplos de associações entre objetos.

Associações possuem diversas características importantes: multiplicidades, nome, direção de leitura, papéis, tipo de participação e conectividade. Nas próximas seções, descrevemos essas características.

5.2.2.1 Multiplicidades

As associações permitem representar a informação dos limites inferior e superior da *quantidade de objetos* aos quais outro objeto pode estar associado. Esses limites são chamados de *multiplicidades* na terminologia da UML.[1] Cada associação em um diagrama de classes possui duas multiplicidades, uma em cada extremo da linha que a representa. Os símbolos possíveis para representar uma multiplicidade estão descritos na Tabela 5-1. Note que, em dois dos casos apresentados nessa Tabela, a UML define mais de um símbolo para representar a mesma multiplicidade. Primeiramente, o símbolo "1" é equivalente à utilização do símbolo "1..1". Outra equivalência ocorre entre os símbolos '*' e '0..*'.

Tabela 5-1: Simbologia para representar multiplicidades

Nome	Simbologia
Apenas Um	1
Zero ou Muitos	0..*
Um ou Muitos	1..*
Zero ou Um	0..1
Intervalo Específico	$l_i..l_s$

[1] As multiplicidades são bastante semelhantes ao conceito de cardinalidade encontrado em outras notações para modelagem de dados (por exemplo: a notação de Peter Chen para o *diagrama de entidades e relacionamentos* bastante conhecido na comunidade de Bancos de Dados).

Como exemplo, observe a Figura 5-4, que exibe duas classes, *Pedido* e *Cliente*, e uma associação entre as mesmas. A leitura dessa associação nos informa que pode haver um objeto da classe `Cliente` que esteja associado a vários objetos da classe `Pedido` (isso é representado pela parte * do símbolo 0..*). Além disso, essa mesma leitura nos informa que pode haver um objeto da classe `Cliente` que não esteja associado a pedido algum (isso é representado pela parte 0 do símbolo 0..*). O diagrama da Figura 5-4 também representa a informação de que um objeto da classe `Pedido` está associado a um, e somente um, objeto da classe `Cliente`.

Figura 5-4: Exemplo de utilização dos símbolos de multiplicidade.

Em todos os símbolos nos quais aparece o '*', este denota que não há um limite superior predefinido para a quantidade máxima de objetos com os quais outro objeto pode se associar. Por exemplo, na Figura 5-4, não há um limite superior, pelo menos na teoria, para a quantidade de pedidos que um cliente pode realizar. Logicamente, a tempo de execução do sistema que está sendo modelado, essa quantidade máxima será sempre limitada pela quantidade de memória do dispositivo computacional no qual a aplicação está executando. No entanto, quando utilizamos o símbolo '*' em um diagrama de classes, queremos dizer que a quantidade máxima de associações é um valor finito maior ou igual a zero, e que não nos importa qual é esse valor.

A Tabela 5-1 também exibe a forma geral do símbolo de multiplicidade para intervalos específicos. As expressões l_i e l_s devem ser substituídas por valores correspondentes aos limites inferior e superior, respectivamente, do intervalo que se quer representar. Por exemplo, vamos representar informações sobre velocistas e corridas nas quais eles participem. Suponha ainda que, em uma corrida deve haver, no máximo, 6 velocistas participantes. O fragmento de diagrama de classes que representa esta situação é exibido na Figura 5-5. O valor para l_i é 2 (uma corrida está associada a, no mínimo, 2 velocistas), e o valor para l_s é 6 (uma corrida está associada a, no máximo, 6 velocistas).

Uma lista de intervalos também pode ser especificada na multiplicidade de uma associação. Por exemplo, a multiplicidade "1, 3, 5..9, 11" é perfeitamente válida, e significa que um objeto pode se associar a uma quantidade de objetos que esteja no conjunto {1, 3, 5, 6, 7, 8, 9, 11}. Note também que, por convenção,

Figura 5-5: Exemplo de utilização de intervalo de valores para multiplicidade.

os valores especificados em uma multiplicidade estão sempre em ordem crescente, quando lidos da esquerda para a direita.

Existem infinitas possibilidades de associação entre objetos (todas as combinações possíveis entre os números inteiros). No entanto, essas associações podem ser agrupadas em apenas três tipos: "*muitos para muitos*", "*um para muitos*" e "*um para um*". Denomina-se *conectividade* o tipo de associação entre duas classes. A conectividade da associação entre duas classes depende dos símbolos de multiplicidade que são utilizados na associação. A Tabela 5-2 exibe as correspondências entre os tipos de conectividade e os símbolos de multiplicidade.

Tabela 5-2: Conectividades *versus* multiplicidades

Conectividade	Multiplicidade de um extremo	Multiplicidade do outro extremo
Um para um	0..1 ou 1	0.. 1 ou 1
Um para muitos	0..1 ou 1	* ou 1..* ou 0..*
Muitos para muitos	* ou 1..* ou 0..*	* ou 1..* ou 0..*

A Figura 5-6 apresenta três exemplos, um para cada conectividade possível. No primeiro (um para um), indica-se que um empregado pode gerenciar um e somente um departamento. Um departamento, por sua vez, possui um único gerente. No segundo exemplo (um para muitos), um empregado está lotado em um único departamento, mas um departamento pode ter diversos empregados. Finalmente, no terceiro exemplo (muitos para muitos), um projeto pode ter diversos empregados como trabalhadores. Além disso, um empregado pode trabalhar em diversos departamentos.

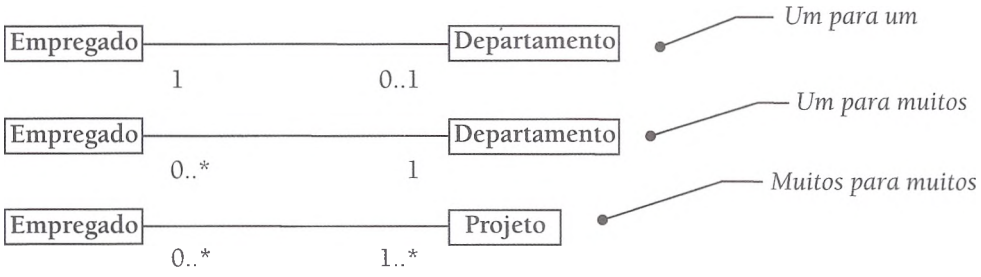


Figura 5-6: Exemplos de cada tipo de conectividade.

5.2.2.2 Participações

Uma característica importante de uma associação está relacionada à necessidade ou não da existência dessa associação entre objetos. Essa característica é denominada *participação*. A participação pode ser *obrigatória* ou *opcional*. Se o va-

lor mínimo da multiplicidade de uma associação é igual a 1 (um), significa que a participação é obrigatória. Caso contrário, a participação é opcional.

Por exemplo, considere a associação entre `Empregado` e `Departamento` na parte superior da Figura 5-6. A multiplicidade de valor 1 próxima a `Empregado` indica que um objeto da classe `Departamento` só pode existir se estiver associado a um objeto `Empregado`. Ou seja, para objetos da classe `Departamento`, a participação é *obrigatória*. No entanto, para objetos de `Empregado`, essa mesma associação é *parcial* (pode haver empregados que não estejam associados a um departamento). Isso é indicado pelo símbolo 0..1 no extremo próximo à classe `Departamento`.

Deve-se ter cuidado com a utilização de participações obrigatórias, pois estas "engessam" o modelo resultante, o que dificulta futuras extensões do mesmo. Se há dúvidas sobre as multiplicidades de uma associação, pode-se adiar essa decisão para mais tarde, quando mais informações sobre o sistema forem conhecidas. Em partículas, as informações obtidas com a *modelagem dinâmica* do sistema servem para definir ou validar as multiplicidades das associações.

5.2.2.3 Nome de associação, direção de leitura e papéis

Para melhor esclarecer o significado de uma associação no diagrama de classes, a UML define três recursos de notação: *nome de associação*, *direção de leitura* e *papel*. O nome da associação é posicionado na linha da associação, a meio caminho das classes envolvidas.

A direção de leitura indica como a associação deve ser lida. Essa direção é representada por um pequeno triângulo posicionado próximo a um dos lados do nome da associação. O nome de uma associação deve fornecer algum significado semântico à mesma.

Quando um objeto participa de uma associação, ele tem um papel específico nela. Uma característica complementar à utilização de nomes e de direções de leitura é a indicação de *papéis* (*roles*) para cada uma das classes participantes em uma associação.

A Figura 5-7 apresenta uma associação na qual são representados os papéis (`contratante` e `contratado`) e o seu nome (`Contrata`). É também representada a

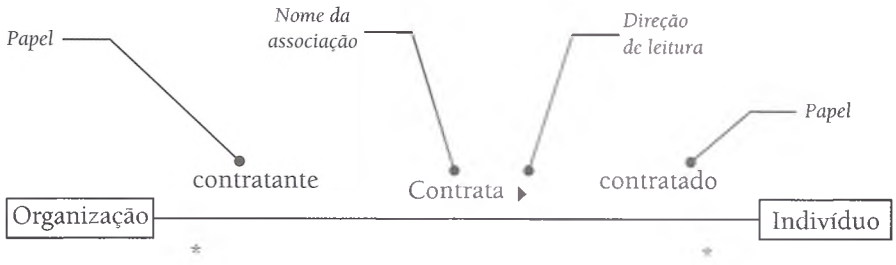


Figura 5-7: Exemplo de utilização de nome de associação, direção de leitura e de papéis.

direção de leitura, que indica que uma organização contrata indivíduos, e não o contrário.

Sempre que o significado de uma associação não for fácil de inferir, é recomendável representar papéis ou pelo menos o nome da associação. Isso evita que haja interpretações erradas no significado da associação. Além disso, essa prática aumenta a legibilidade do diagrama. O nome da associação deve ser simples e exprimir o significado da mesma. É preferível não nomear associações a usar nomes vagos ou óbvios demais. O mesmo vale para os papéis: em situações em que o significado da associação for intuitivo, a utilização de papéis só serve para "carregar" o diagrama. O ponto é tentar equilibrar clareza e concisão.

Embora não seja usual, pode haver diversos motivos pelos quais um objeto precisa saber sobre outro. Conseqüentemente, pode haver diversas associações definidas entre duas classes no diagrama de classes. Por exemplo, considere duas classes: `Empregado` e `Departamento`. Considere, ainda, que um departamento precisa saber quais são seus empregados e quem é o seu gerente. Nesse caso, há duas associações diferentes, embora as classes envolvidas sejam as mesmas (ver Figura 5-8).

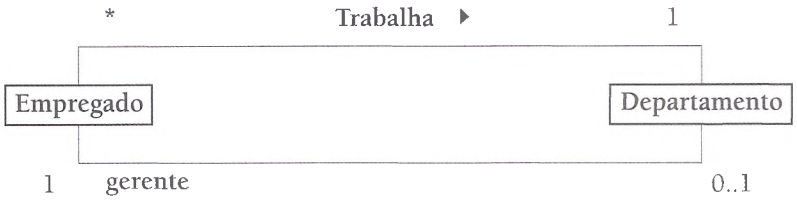


Figura 5-8: Associações diferentes entres duas classes.

5.2.2.4 Classes associativas

Classes associativas são classes que estão ligadas a associações, em vez de estarem ligadas a outras classes. São também chamadas de *classes de associação*. Esse tipo de classe normalmente aparece quando duas ou mais classes estão associadas, e é necessário manter informações sobre a associação existente entre as mesmas. Embora seja mais comum encontrar classes associativas ligadas a associações de conectividade *muitos para muitos*, uma classe associativa pode estar ligada a associações de qualquer conectividade.

Na UML, uma classe associativa é representada pela mesma notação utilizada para uma classe comum. A diferença é que esta classe é ligada por uma linha tracejada a uma associação. Para ilustrar a utilização de uma classe associativa em um diagrama de classes, apresentamos a Figura 5-9. Esse diagrama informa que uma pessoa trabalha como empregado em várias empresas. Uma empresa, por sua vez, tem vários empregados. A classe associativa `Emprego` permite saber, para cada par de objetos [empregado, empregador], qual o salário e a data de

contratação do empregado em relação àquele empregador. De forma geral, se precisarmos representar a existência de uma informação que somente faz sentido quando consideramos todos os objetos participantes da associação, podemos utilizar o conceito de classe associativa.

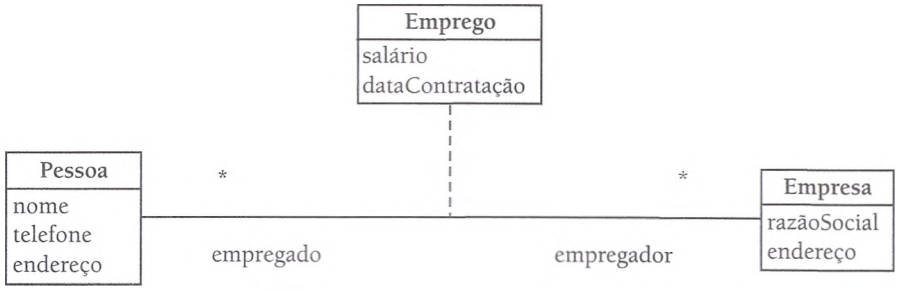


Figura 5-9: Exemplo de classe associativa.

Como dica de modelagem, não se deve nomear a linha da associação de uma classe associativa. Nesse caso, somente o nome escolhido para a classe associativa deve ser suficiente para expressar o significado desejado. Alternativamente, papéis (ver Seção 5.2.2.3) também podem ser utilizados para dar significado à associação.

Note que uma classe associativa pode participar de outros relacionamentos. Por exemplo, observe o diagrama da Figura 5-10, no qual podemos verificar que há funcionários com várias especialidades. Consertos em automóveis são realizados por funcionários, mas é necessário saber que especialidade foi utilizada pelo funcionário em certo conserto. Para isso, uma classe associativa é criada entre as classes `Funcionário` e `Automóvel`. Além disso, essa mesma classe associativa está associada à classe especialidade para permitir conhecer qual a especialidade utilizada em um conserto.

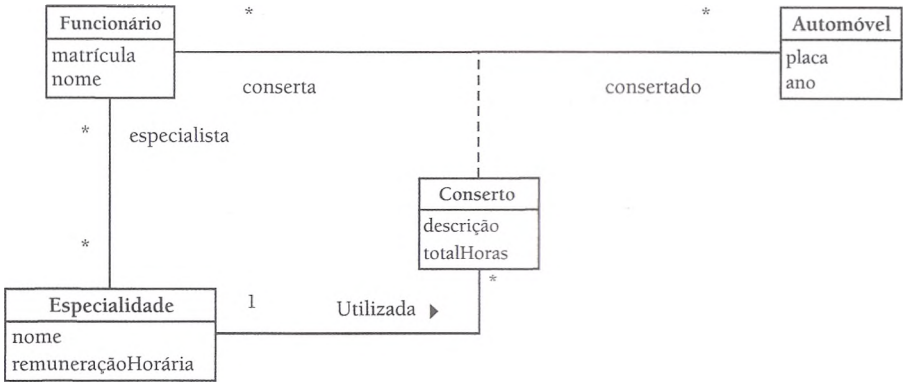


Figura 5-10: Exemplo de classe associativa.

Pelos exemplos anteriores, pode-se notar que uma classe associativa é um elemento híbrido: tem características de uma classe, mas também características de uma associação.

Por fim, de forma geral, um diagrama de classes que contém uma classe associativa pode ser modificado para retirá-la, sem perda de informação no modelo em questão. Isso pode ser feito em dois passos: (1) eliminação da associação correspondente à classe associativa e (2) criação de associações diretas desta última com as classes que antes eram conectadas pela associação eliminada no passo 1. Como exemplo de aplicação desses passos, a Figura 5-11 apresenta um diagrama equivalente ao da Figura 5-9, em que a classe associativa foi substituída por uma classe ordinária. Note que a classe Emprego tem participação obrigatória em ambas as associações.

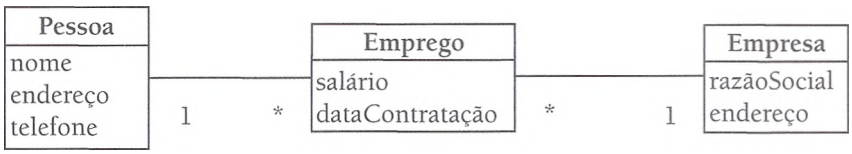


Figura 5-11: Uma classe associativa pode ser substituída por uma classe ordinária.

5.2.2.5 Associações ternárias

Define-se o *grau* de uma associação como a quantidade de classes envolvidas na mesma. Na grande maioria dos casos práticos de modelagem, as associações normalmente são binárias, ou seja, representam a ligação entre objetos de duas classes (tem grau igual a dois). Entretanto, de forma geral, uma associação pode ter grau maior que dois. Quando o grau de uma associação é igual a três, dizemos que a mesma é ternária. Associações ternárias são necessárias quando é preciso associar três objetos distintos. A Figura 5-12 ilustra a notação da UML para associações ternárias.

Dada uma associação ternária (ou de mais alto grau) entre as classes A, B e C, a multiplicidade do extremo C indica quantos objetos da classe C podem estar associados com um par particular de objetos (*b, c*), em que *b* é um objeto da classe *B*, e *c* é uma instância da classe *C*.

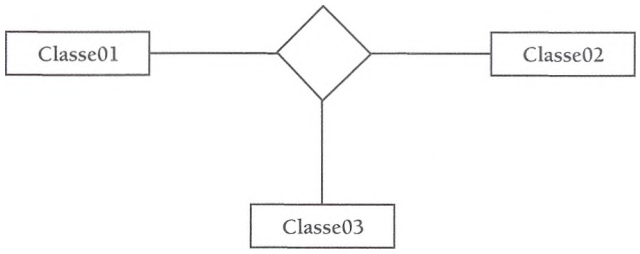


Figura 5-12: Notação para representar associações ternárias no diagrama de classes.

Um exemplo de associação ternária é apresentado na Figura 5-13. Esta figura ilustra o fato de que um técnico utiliza exatamente um computador para cada projeto em que trabalha. Cada computador pertence a um técnico para cada projeto. Note que um técnico pode trabalhar em muitos projetos e utilizar diferentes computadores para diferentes projetos.

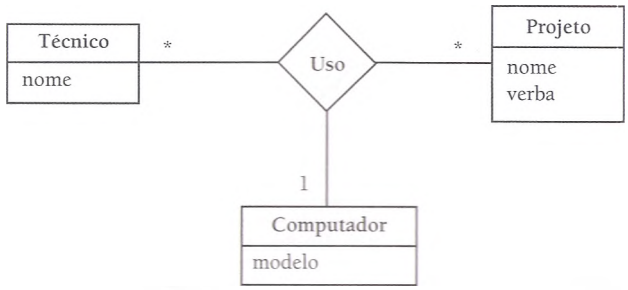


Figura 5-13: Exemplo de associação ternária.

Outro exemplo de associação ternária é apresentado na Figura 5-14. Desta vez, vê-se que cada empregado associado a um projeto trabalha em somente uma localização, mas pode estar em diferentes localizações em projetos diferentes. Em uma localização em particular, pode haver muitos empregados associados a um dado projeto.

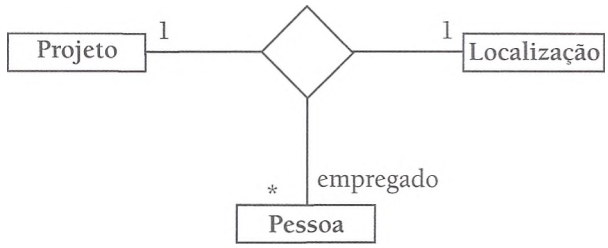


Figura 5-14: Exemplo de associação ternária.

Finalmente, os conceitos de classe associativa e de associação ternária podem ser misturados no conceito de *classe associativa ternária*, no qual existe uma classe associativa ligada a uma associação ternária. A Figura 5-15 ilustra um exemplo desta situação. Há três classes ligadas por uma associação ternária (`Estudante`, `Instrutor` e `Curso`). Além disso, existe a classe associativa `SeçãoCurso`, que permite armazenar informações para cada instância da associação.

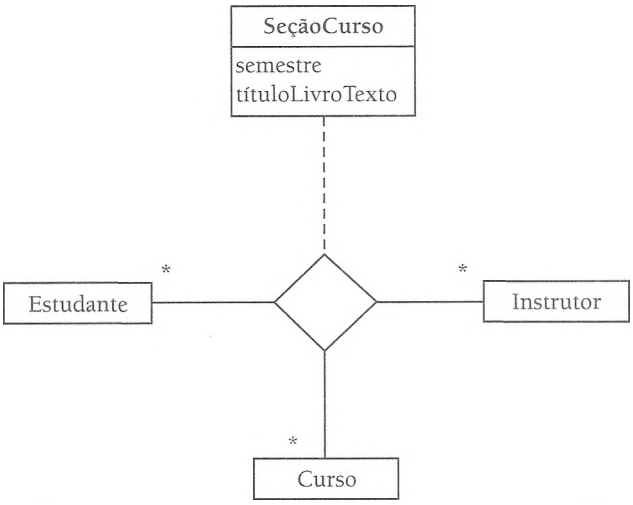


Figura 5-15: Exemplo de classe associativa ternária.

5.2.2.6 Associações reflexivas

Uma associação reflexiva (também denominada *auto-associação*) associa objetos da mesma classe. Cada objeto tem um papel distinto nessa associação. Por exemplo, considere a Figura 5-16, em que existe uma associação reflexiva entre objetos de `Empregado`. Nessa figura, apresentamos um exemplo de uso de auto-associação, em que há objetos que assumem o papel de `supervisor` e outros objetos que assumem o papel de `supervisionado`. Nesse exemplo, o nome da associação poderia ter sido omitido, uma vez que os papéis foram definidos (ver Seção5.2.2.3). Além disso, conforme mostra a Figura 5-16, em associações reflexivas, a utilização de papéis é bastante importante para evitar qualquer confusão na leitura da associação.

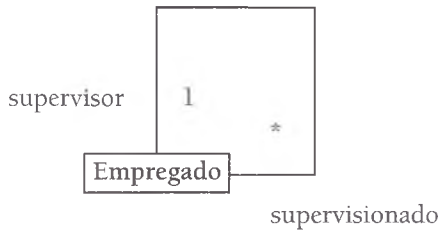


Figura 5-16: Exemplo de associação reflexiva.

Não se deve confundir o significado de uma associação reflexiva. Ela *não* indica que um objeto se associa a ele próprio (um empregado não é supervisor dele próprio; uma disciplina não é pré-requisito dela mesma etc.). Em vez disso, uma auto-associação indica que um objeto de uma classe se associa com outros objetos da mesma classe.

5.2.2.7 Agregações e composições

A toda associação podemos atrelar uma *semântica*. A semântica de uma associação corresponde ao seu significado, ou seja, à natureza conceitual da relação que existe entre os objetos que participam daquela associação. Quando damos um nome a uma associação (ou alternativamente definimos os papéis dos objetos associados pela mesma), fazemos isso com o objetivo de esclarecer a sua semântica.

De todos os significados diferentes que uma associação pode ter, há uma categoria especial de significados, que representa relações *todo-parte*. Uma relação todo-parte entre dois objetos indica que um dos objetos *está contido* no outro. Podemos também dizer que um objeto *contém* o outro.

A UML define dois tipos de *relacionamentos todo-parte*, a *agregação* e a *composição*. A agregação e a composição são casos especiais da associação. Conseqüentemente, todas as características válidas para esta última (multiplicidades, participações, papéis etc.) valem para as primeiras. Além disso, sempre que for possível utilizar uma agregação ou composição, uma associação também poderá ser utilizada. De qualquer modo, a seguir são relacionadas algumas características particulares das agregações e composições que as diferem das associações simples.

- Agregações/composições são *assimétricas*, no sentido de que, se um objeto A é parte de um objeto B, o objeto B não pode ser parte do objeto A.
- Agregações/composições propagam comportamento, no sentido de que um comportamento que se aplica a um todo automaticamente se aplica às suas partes.
- Nas agregações/composições, as partes são normalmente criadas e destruídas pelo todo. Na classe do objeto todo, são definidas operações para adicionar e remover as partes.

Em uma situação prática, podemos aplicar a seguinte regra para verificar se faz sentido utilizarmos um relacionamento todo-parte (agregação ou composição). Sejam duas classes associadas, X e Y. Se uma das perguntas a seguir for respondida com um sim, provavelmente há um relacionamento todo-parte envolvendo X e Y, no qual X é o todo, e Y é a parte. (Do contrário, é melhor utilizarmos a associação simples.)

1) X tem um ou mais Y?
2) Y é parte de X?

Graficamente, a UML fornece notações diferentes para a agregação e a composição. Uma agregação é representada como uma linha que conecta as classes relacionadas, com um diamante (losango) branco perto da classe que represen-

ta o todo. Já uma composição é representada na UML por meio de um diamante negro, para contrapor com o diamante branco da agregação.

Como exemplo de agregação, analise a Figura 5-17, que apresenta um fragmento de diagrama de classes. Esse diagrama indica que uma associação esportiva é formada por diversas equipes. Cada equipe é formada por diversos jogadores. Por outro lado, um jogador pode fazer parte de diversas equipes.

Figura 5-17: Exemplo de agregação.

Como exemplo de composição, considere os itens de um pedido de compra. É comum um pedido de compra incluir vários itens. Cada item diz respeito a um produto faturado. Os itens têm identidade própria (é possível distinguir um item de outro no mesmo pedido). Essa situação é representada no diagrama da Figura 5-18. Outro exemplo de composição é apresentado na Figura 5-19. Esse diagrama informa que um automóvel possui várias partes, a saber, um motor e quatro rodas.

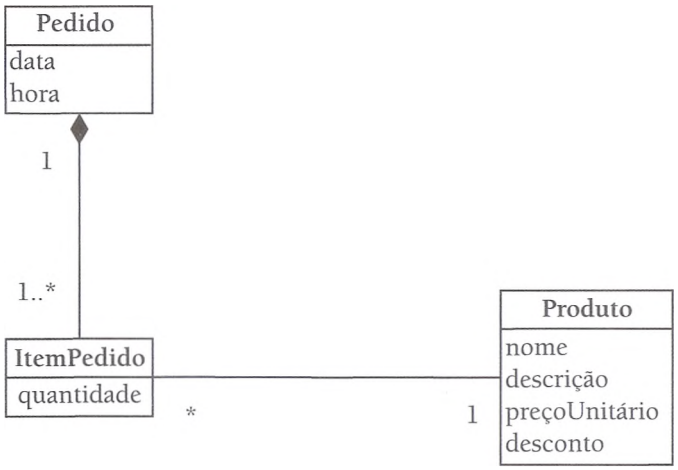


Figura 5-18: Composição entre `Pedido` e `ItemPedido`.

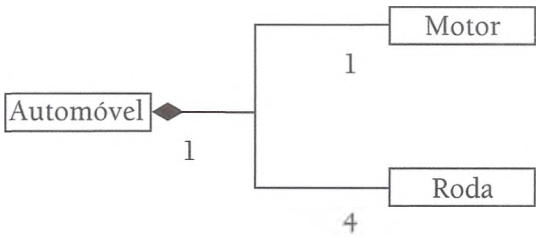


Figura 5-19: Exemplo de utilização de composição. Todo: Automóvel. Partes: Motor e Roda.

Uma agregação pode se estender por diversos níveis. Isso significa que pode haver um objeto A que seja composto de objetos B, e que este último seja composto por objetos C, e assim por diante. Isso gera uma *hierarquia de agregações*. O exemplo da Figura 5-17, em que há dois níveis de agregação, permite notar essa característica. Assim como as agregações, composições também podem se estender por diversos níveis. Dessa forma, pode-se falar em uma *hierarquia de composições*. Um exemplo de hierarquia de composição é a Figura 5-20, que mostra que capítulos são compostos de seções, estas, por sua vez, são compostas de parágrafos.

Figura 5-20: Exemplo de hierarquia de composição.

Se um modelador constata que uma associação tem uma semântica todo-parte, ele deve decidir que relacionamento utilizar, se uma agregação ou uma composição. Infelizmente, tanto na prática, quanto na teoria, as diferenças entre a agregação e a composição não são bem definidas. Mesmo assim, a seguir descrevemos as duas diferenças mais marcantes entre os dois tipos de relacionamentos todo-parte.

1. Na agregação, a destruição de um objeto todo não implica necessariamente a destruição do objeto parte. Por exemplo, considere a Figura 5-17. Se uma das equipes das quais um jogador é membro for extinta por algum motivo, este jogador ainda poderá continuar membro de outras equipes. Por outro lado, considere o exemplo da Figura 5-18. Nessa situação, os itens não têm existência independente do pedido ao qual estão conectados. Quando o pedido deixa de existir, o mesmo acontece com os seus itens.
2. Na composição, os objetos *parte* pertencem a um único *todo*. Por essa razão, a composição é também denominada *agregação não-compartilhada*. Por outro lado, em uma agregação, pode ser que um mesmo objeto participe como componente de vários outros objetos. Por essa razão, a agregação é também denominada *agregação compartilhada*.

5.2.2.8 Restrições sobre associações

Restrições podem ser adicionadas sobre uma associação para adicionar a ela mais semântica. Duas das restrições sobre associações predefinidas pela UML são `subset` (subconjunto) e `xor` (ou exclusivo). Ambas são representadas por uma linha pontilhada que liga duas ou mais associações. Embora a descrição a

seguir faça referência a associações, essas restrições também podem ser utilizadas em agregações e composições.

A restrição subset indica que os objetos conectados por uma associação constituem um subconjunto dos objetos conectados através de uma outra associação. A Figura 5-21 ilustra o uso da restrição subset. Neste exemplo, há duas classes ligadas pela restrição subset, indicando que os objetos associados através de Administra formam um subconjunto dos objetos associados através de Reside. (Portanto, a flecha parte da associação correspondente ao subconjunto.)

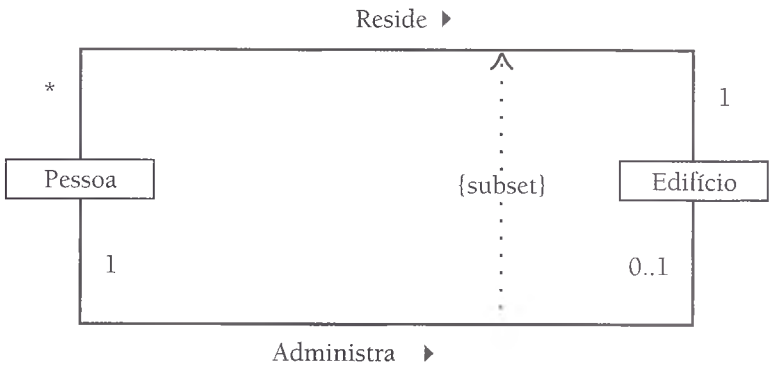


Figura 5-21: Exemplo de utilização da restrição subset.

Na restrição xor, há duas ou mais classes ligadas pela linha pontilhada. Essas classes devem ter associações com uma classe em comum. Essa restrição significa que somente uma das associações envolvidas pode ocorrer entre os objetos. Como exemplo da restrição xor, considere a Figura 5-22 (por simplificação, atributos e operações são omitidos). Este exemplo indica que um objeto ContaBancária pode estar associado a objetos Pessoa ou a objetos Instituição, mas nunca a objetos dos dois tipos simultaneamente.

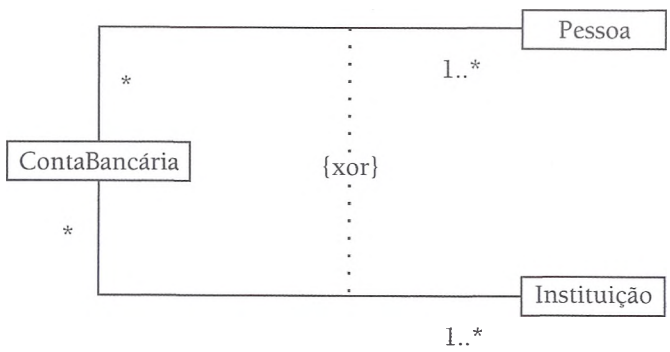


Figura 5-22: Exemplo de utilização da restrição *xor*.

As restrições também podem ser definidas formalmente em OCL (ver Seção 3.6). Expressões nessa linguagem podem ser definidas utilizando-se uma expressão da forma Item.seletor. Essa expressão permite ter acesso às propriedades de uma classe (atributos, operações e associações). Essas expressões podem ser combinadas recursivamente para formar expressões mais complexas. Além disso, os operadores aritméticos (+, -, *, /) e lógicos (>, .=, <, ,=, < >, =) também estão disponíveis para definir expressões na OCL. A seguir, descreveremos vários exemplos de diagramas de classes nos quais são definidas expressões em OCL.

Na Figura 5-23, a expressão em OCL define que, para um objeto Vôo, a quantidade de horas de treinamento do piloto deve ser maior ou igual à quantidade mínima de horas exigida para o avião a ser pilotado.

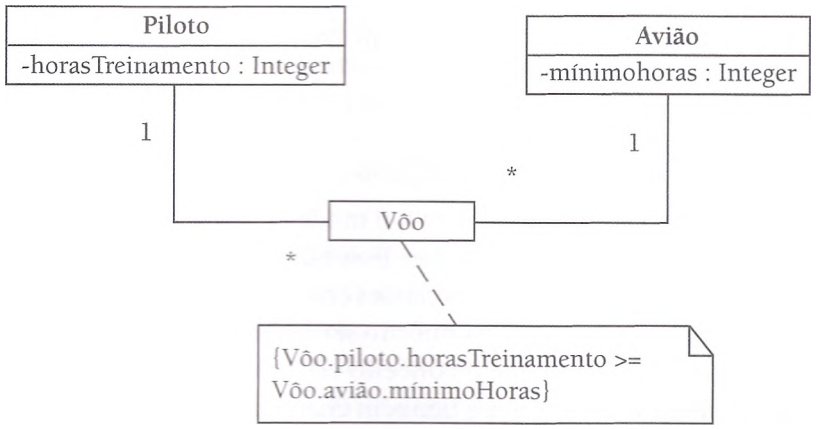


Figura 5-23: Exemplo de utilização da OCL para definir restrições.

No exemplo da Figura 5-24, tem-se uma restrição em OCL definida sobre o elemento de associação entre as classes do diagrama. Essa restrição indica que um indivíduo deve ter mais de 21 anos para participar de uma sociedade.

Um outro exemplo (adaptado do documento de especificação da UML) é ilustrado na Figura 5-25, em que há duas classes, Empresa e Pessoa. A restri-

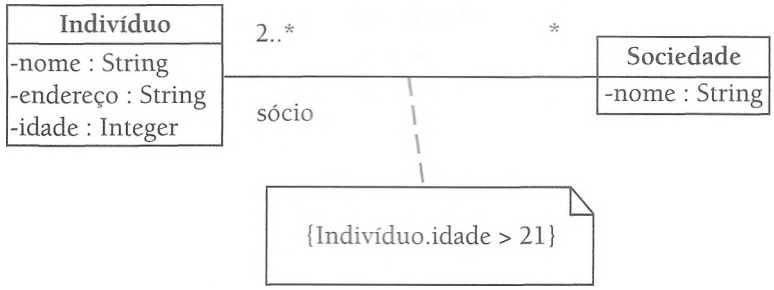


Figura 5-24: Exemplo de definição de restrição em OCL.

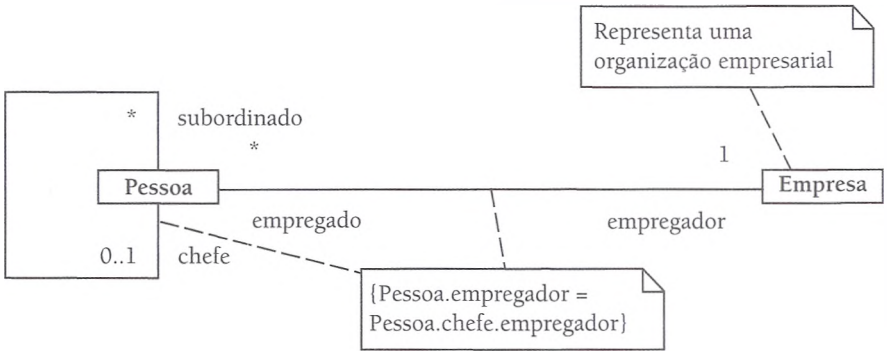


Figura 5-25: Exemplo de definição de restrição em OCL.

ção em OCL deste exemplo significa que uma pessoa que trabalhe para uma empresa empregadora e o gerente dessa pessoa devem trabalhar para a mesma empresa.

5.2.3 Generalizações e especializações

Além de relacionamentos entre objetos, o modelo de classes também pode representar relacionamentos entre classes. Esses últimos denotam relações de generalidade ou especificidade entre as classes envolvidas. Por exemplo, o conceito *mamífero* é mais genérico que o conceito *ser humano*. Outro exemplo: o conceito *carro* é mais específico que o conceito *veículo*.

O relacionamento de herança é também chamado de relacionamento de *generalização/especialização*, ou simplesmente *gen/espec*. Isso porque a generalização e a especialização são dois pontos de vista do mesmo relacionamento: dadas duas classes A e B, se A é uma generalização de B, então B é uma especialização de A.

Os termos para denotar o relacionamento de herança são bastante variados. O termo *subclasse* é utilizado para denotar a classe que herda as propriedades de outra classe através de uma generalização. Diz-se, ainda, que a classe que possui propriedades herdadas por outras classes é a *superclasse*. Utilizam-se ainda os nomes *supertipo* e *subtipo* como sinônimos para superclasse e subclasse, respectivamente. Outros termos (mais utilizados em linguagens de programação orientada a objetos) são *classe base* (sinônimo para superclasse) e *classe herdeira* (sinônimo para subclasse). Diz-se também que uma subclasse é uma *especialização* de sua superclasse (a subclasse especializa a superclasse), e que uma superclasse é uma *generalização* de suas subclasses (a superclasse generaliza as subclasses). Os termos *ancestral* e *descendente* fazem referência ao uso do relacionamento de generalização em vários níveis (ver Seção 5.2.3.1).

No diagrama de classes, a herança é representada na UML por uma flecha partindo da subclasse em direção à superclasse. Alternativamente, podemos

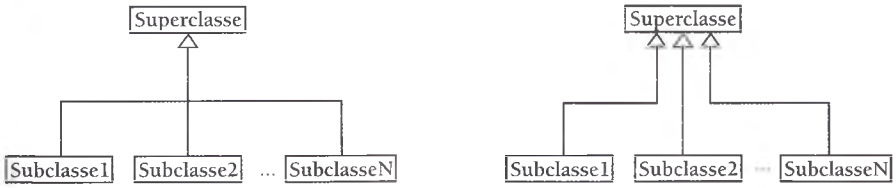


Figura 5-26: Representações alternativas para o relacionamento de generalização na UML.

juntar as flechas de todas as subclasses de uma classe, como mostra a Figura 5-26. A utilização de um ou de outro estilo é questão de gosto.

Para entender a semântica de um relacionamento de herança, é preciso lembrar que uma classe representa um conjunto de objetos que partilham um conjunto comum de propriedades (atributos, operações, associações). No entanto, alguns objetos, embora bastante semelhantes a outros, podem possuir um conjunto de propriedades que outros não possuem. Por exemplo, todos os funcionários de uma empresa partilham os atributos `nome`, `endereço`, `númeroMatrícula` e `salárioBase`. No entanto, um tipo especial de funcionário – o dos vendedores – possui atributos específicos, por exemplo, `zonaGeográfica` (zona em que trabalham) e `taxaComissão` (porcentagem de comissão que recebem pelas vendas realizadas). Os vendedores pertencem à classe dos funcionários, mas eles por si próprios formam uma outra classe, a dos vendedores. Portanto, vendedores possuem todas as características inerentes a funcionários, além de possuírem características próprias. Diz-se, assim, que a classe de vendedores *herda* propriedades da classe de funcionários.

É importante notar que não somente atributos e operações são herdados pelas subclasses, mas também as associações que estão definidas na superclasse. A Figura 5-27 é um exemplo dessa propriedade: existe uma associação entre `Cliente` e `Pedido`. Não há necessidade de se criar uma associação entre `Pedido` e `ClientePessoaFísica` e entre `Pedido` e `ClientePessoaJurídica`, pois as subclasses herdam a associação de sua superclasse. Por outro lado, se existe uma associação que não é comum a todas as subclasses, mas, sim, particular a algu-

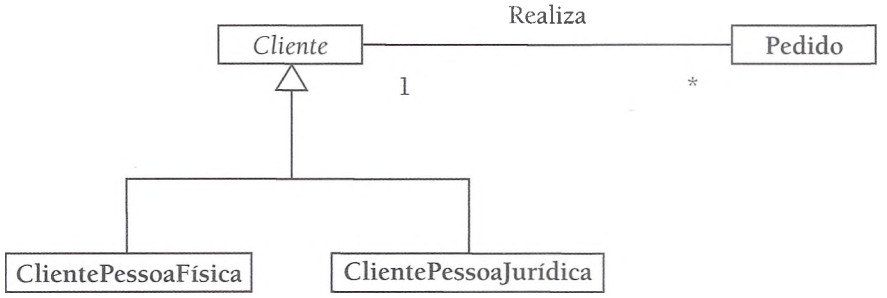


Figura 5-27: Não há necessidade de se criar uma associação entre Pedido e as subclasses de Cliente.

mas subclasses, então essa associação deve ser representada em separado, envolvendo somente as subclasses em questão.

Um conceito que pode ser utilizado em situações de modelagem é a *classe abstrata*. Uma classe abstrata normalmente é utilizada para organizar a hierarquia de classes. Uma classe abstrata é utilizada para fins de modelagem como um contêiner de definições de propriedades. Classes abstratas não geram objetos diretamente (embora objetos de uma subclasse de uma classe abstrata sejam também considerados objetos dessa última). Na notação da UML, uma classe abstrata é representada com seu nome em *itálico*. Por exemplo, na Figura 5-27, a classe `Cliente` é `abstrata`. As classes `Conta` e `ContaBancária` na Figura 5-28 também são abstratas. O conceito de classe abstrata tem fundamental importância no contexto da modelagem de classes de projeto. Na Seção 8.5.2, descrevemos o conceito de classe abstrata naquele contexto.

É importante notar que o relacionamento de herança difere do relacionamento de associação, porque o primeiro se trata de um relacionamento *entre classes*. Quando se diz, por exemplo, que gerentes são tipos especiais de funcionários significa que objetos da classe gerente possuem características comuns a todos os funcionários, além de poderem ter características próprias de um gerente. Por outro lado, os relacionamentos de associação, agregação e composição são definidos *entre objetos*. Isso equivale a dizer que objetos específicos de uma classe se associam entre si ou com objetos específicos de outras classes.

O relacionamento de herança acontece entre *classes*. Já o relacionamento de associação acontece entre *instâncias de classes* (objetos).

5.2.3.1 Propriedades do relacionamento de herança

O relacionamento de herança possui duas propriedades importantes, *transitividade* e *assimetria*, que descrevemos a seguir.

A propriedade da *transitividade* indica que uma classe em uma hierarquia herda tanto propriedades e relacionamentos de sua superclasse imediata quanto de suas superclasses não imediatas (classes em um nível mais alto da hierarquia). A propriedade de transitividade entre classes também pode ser vista sob a perspectiva de que toda instância de uma classe também é uma instância de suas superclasses. Ou seja, uma instância de uma classe C também é instância de todos os ancestrais de C. Uma classe é uma *generalização* de outra classe se toda instância desta última for também uma instância da primeira. Pela propriedade de transitividade, a herança pode ser aplicada em vários níveis, ou seja, uma classe que herda propriedades de uma outra classe pode ela própria servir como superclasse. A Figura 5-28 ilustra uma *hierarquia de herança* de três níveis. Nes-

sa hierarquia, são apresentadas cinco classes. Note que `ContaPoupança` possui um atributo próprio, e mais três atributos herdados de `Conta`. Note também a disposição das classes no diagrama, no qual as classes mais genéricas estão posicionadas acima das classes mais específicas. Essa disposição não é obrigatória, pois o sentido da seta indica qual é a classe mais genérica. No entanto, é recomendado dispor as classes dessa forma para enfatizar a hierarquia existente entre superclasses e subclasses.

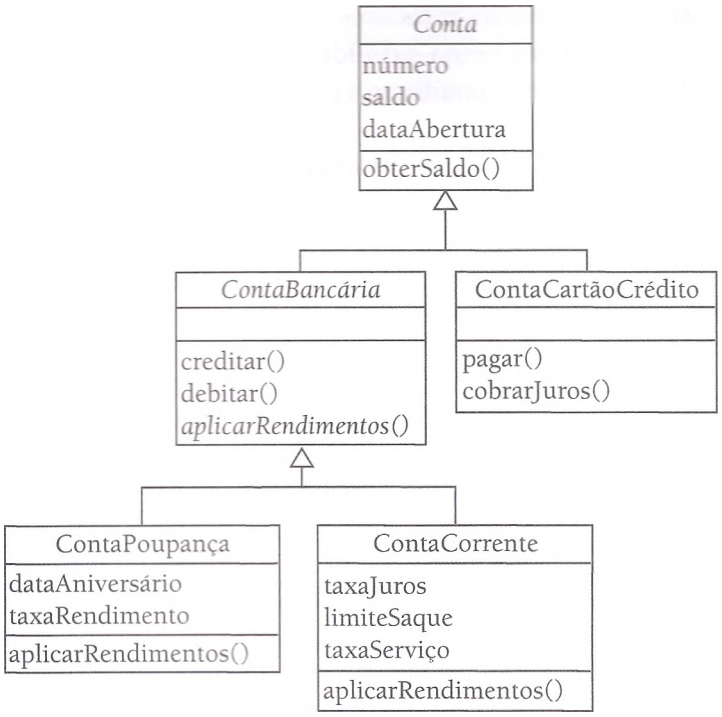


Figura 5-28: Exemplo de hierarquia de herança.

Outra propriedade importante da herança é a *assimetria*. Essa propriedade significa que dadas duas classes A e B, se A for uma generalização de B, então B não pode ser uma generalização de A. Ou seja, *não* pode haver ciclos em uma hierarquia de herança.

5.2.3.2 Refinando o modelo de classes com gen/espec

Graças a relacionamentos de herança, podem ser feitos refinamentos no modelo de classes. A idéia básica é identificar abstrações mais genéricas ou mais específicas que outras no diagrama de classes do sistema. Essa identificação de abstrações corresponde a refinamentos no modelo de classes que podem ser obtidos a partir de duas estratégias alternativas e complementares: a *generalização* e a *especialização*.

Primeiro pode ser que duas ou mais classes semelhantes tenham sido identificadas. Neste caso, pode ser adequado criar uma *generalização*, ou seja, uma classe mais genérica, e definir as classes anteriores como subclasses desta última. Pode ser que a superclasse não gere instâncias próprias e esteja sendo utilizada somente para organizar classes semelhantes em uma hierarquia. Neste caso, a superclasse pode ser definida como *abstrata* (ver Seção 8.5.2), o que significa que ela tem o objetivo de organizar o modelo.

Em segundo lugar, pode-se também aplicar a *especialização*, que corresponde ao processo de criar classes mais específicas a partir de uma classe preexistente. Por exemplo, talvez tenha sido identificada uma nova propriedade de uma classe preexistente que justifique a criação de uma subclasse na qual esta propriedade seja definida.

Seja qual for o processo de derivação utilizado, generalização ou especialização, uma dica prática para confirmarmos se há legitimamente um relacionamento de gen/espec entre duas classes é aplicar a chamada *regra da substituição* que apresentamos a seguir. (Por conta dessa regra, o relacionamento gen/espec é também conhecido como *relacionamento é-um*.)

Regra da substituição: sejam duas classes A e B, onde A é uma generalização de B. Não pode haver diferenças entre utilizar instâncias de B ou de A, do ponto de vista dos clientes de A.

Uma conseqüência da regra da substituição é a seguinte: é inadequado o uso do relacionamento gen/espec onde nem todas as propriedades e comportamentos da superclasse fazem sentido para a subclasse.

Outro teste que pode ser realizado para identificar se duas classes se relacionam por gen/espec, e que é equivalente ao da regra da substituição, é perguntar o seguinte: *X é um tipo de Y?* Se a resposta for "sim", provavelmente a classe X deve ser definida como uma subclasse de Y.

Deve-se também considerar a conformidade da subclasse em relação às propriedades definidas em sua superclasse. Ou seja, é inadequado o uso de gen/espec na situação em que nem todas as propriedades da superclasse fazem sentido para a subclasse. Essa restrição é uma interpretação da regra da substituição.

Como um exemplo de uso adequado de gen/espec (e que respeita a regra da substituição), observe o diagrama de classes da Figura 5-29. Note que tanto `ContaCorrente` quanto `ContaPoupança` são tipos de `ContaBancária`. Note também que os relacionamentos e propriedades da superclasse valem para ambas as subclasses.

Através da generalização e da especialização, classes mais gerais podem ser definidas a partir de classes mais específicas, e classes mais específicas também podem

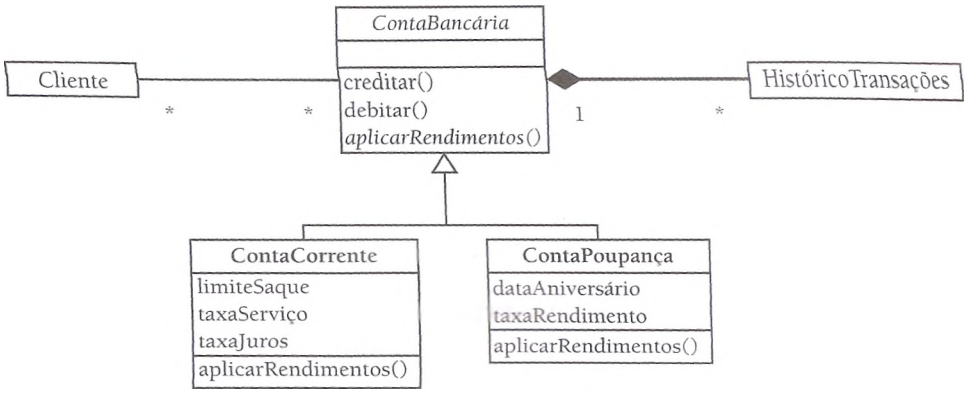


Figura 5-29: Conformidade das subclasses à superclasse.

ser definidas a partir de classes mais gerais. Entretanto, na aplicação de generalização ou especialização, deve-se evitar a construção de hierarquias de herança muito profundas (com mais de três níveis), pois elas dificultam a leitura do diagrama.

Outra dica: papéis (ver Seção 5.2.2.3) e subclasses não devem ser confundidos. Um papel corresponde ao uso de certa classe em uma associação. Uma classe pode assumir vários papéis. O modelador deve evitar a criação de subclasses em situações que podem ser resolvidas através da utilização de papéis. Veja o exemplo da Figura 5-30.

Figura 5-30: Situação em que é mais adequado utilizar um papel do que uma subclasse.

5.2.3.3 Definição de restrições sobre gen/espec

Conforme descrito na Seção 3.4, a UML permite que determinadas restrições sejam associadas a elementos de um modelo. Nesta seção, descreveremos algumas restrições predefinidas pela UML que se aplicam ao relacionamento gen/espec.

As restrições sobre gen/espec são representadas no diagrama de classes, próximas à linha do relacionamento. Essas restrições são apresentadas entre chaves. *As restrições predefinidas pela UML para hierarquias de herança são* descritas na Tabela 5-3.

Tabela 5-3: Restrições predefinidas para generalizações

Restrição	Significado
sobreposta	Posteriormente, podem ser criadas subclasses que herdem de mais de uma subclasse (herança múltipla).
disjunta	Quaisquer subclasses criadas posteriormente poderão herdar de somente uma subclasse.
completa	Todas as subclasses possíveis foram enumeradas na hierarquia.
incompleta	Nem todas as subclasses foram enumeradas na hierarquia.

As duas primeiras restrições (sobreposta e disjunta) e as duas últimas (`completa` e `incompleta`) são exclusivas entre si. Isso quer dizer que não há sentido em definir subclasses que sejam sobrepostas e disjuntas ao mesmo tempo, e que não tem sentido definir subclasses simultaneamente com as restrições completa e incompleta. A seguir, veremos diversos exemplos de hierarquias de herança em que se utilizam restrições.

O exemplo da Figura 5-31 informa que há outras subclasses de `Veículo` além das que foram enumeradas na hierarquia.

Na Figura 5-32, as restrições `completa` e `disjunta` significam que (1) não há objeto dentro desta hierarquia que não seja nem homem nem mulher (completa) e (2) não há objeto nesta hierarquia que seja homem e mulher ao mesmo tempo (disjunção).

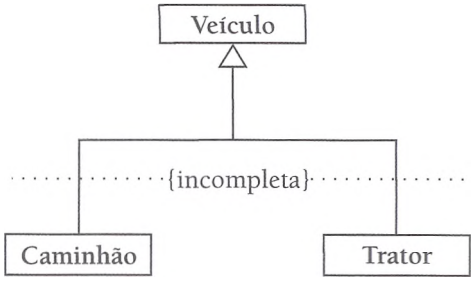


Figura 5-31: Herança incompleta.

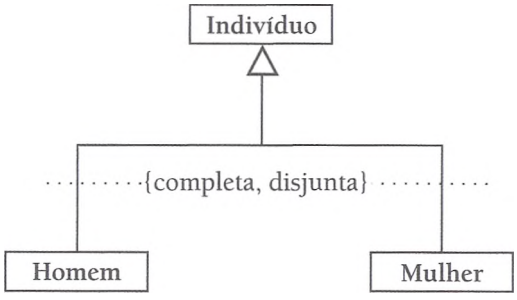


Figura 5-32: Herança completa e disjunta.

Na Figura 5-33, o uso da restrição sobreposta significa que um `Atleta` pode ser tanto `Nadador` quanto `Corredor`. Além disso, há outras subclasses de `Atleta` não enumeradas no diagrama (incompleta).

Na Figura 5-34, `Elipse`, `Quadrado` e `Círculo` são tipos de FiguraGeométrica. Além disso, existem outras subclasses não especificadas na hierarquia.

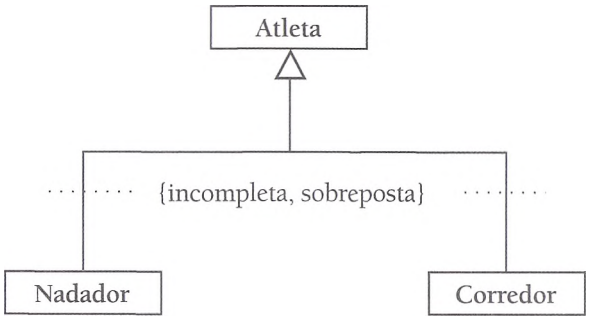


Figura 5-33: Herança incompleta e sobreposta.

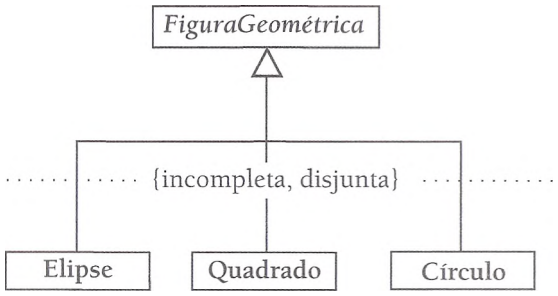


Figura 5-34: Herança incompleta e disjunta.

5.3 Diagrama de objetos

Além do diagrama de classes, a UML define um segundo tipo de diagrama estrutural: o *diagrama de objetos*. Diagramas de objetos podem ser vistos como instâncias de diagramas de classes, da mesma forma que objetos são instâncias de classes.[2] Assim como diagramas de classes, diagramas de objetos são estruturas estáticas. Um diagrama de objetos exibe uma "fotografia" do sistema em certo momento, exibindo as ligações formadas entre objetos conforme estes interagem e de acordo com os valores dos seus atributos.

Em um diagrama de objetos, cada objeto é representado por um retângulo com dois compartimentos. No compartimento superior, a identificação do objeto é exibida. No compartimento inferior (cuja utilização é opcional), aparecem valores para os atributos definidos na classe do objeto.

2. Diagramas de objetos são também chamados *diagramas de instâncias*.

A identificação do objeto deve ser sempre sublinhada. Por convenção, o nome da classe começa com letra maiúscula. O nome do objeto pode ser omitido quando for conveniente. Os dois formatos possíveis para a identificação de um objeto, juntamente com exemplos de cada um, são apresentados na Tabela 5-4.

Tabela 5-4: Possíveis formatos para identificação de instâncias em um diagrama de objetos

Formato	Exemplo
:NomeClasse	:Pedido
nomeObjeto: NomeClasse	umPedido: Pedido

O segundo compartimento do retângulo que representa um objeto, quando utilizado, exibe uma lista de pares da forma *nome do atributo: valor do atributo*.

Como um exemplo, a Figura 5-35 exibe um diagrama de objetos instanciado a partir do diagrama de classes da Figura 5-18. Este diagrama de objetos mostra uma instância de pedido ligada a três instâncias de item de pedido. Cada item, por sua vez, está ligado a um produto. Note que são apresentados valores para cada atributo de um objeto.

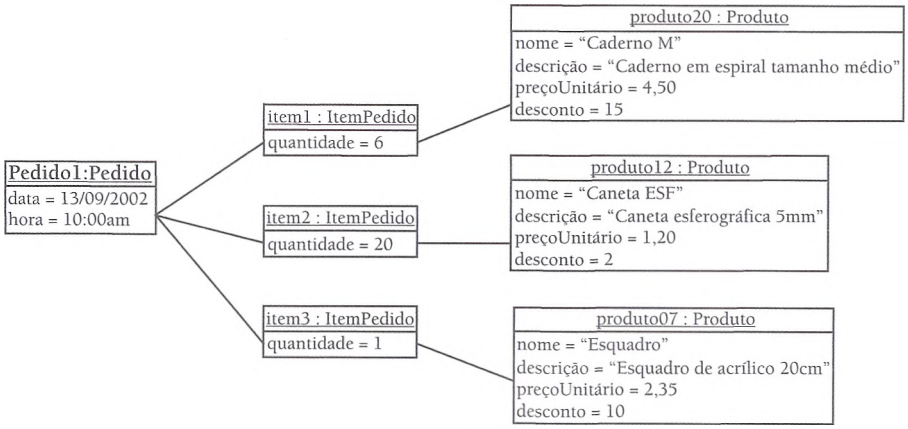


Figura 5-35: Exemplo de diagrama de objetos.

A Figura 5-36 exibe outro exemplo de diagrama de objetos, em que os compartimentos dos valores de atributos são omitidos. Esse diagrama é uma instância do diagrama de classes da Figura 5-16.

O diagrama de objetos é uma contribuição do método de Booch (ver Seção 1.3) à UML. Considerados sozinhos, esses diagramas são raramente utilizados na prática. A única utilidade prática e direta dos diagramas de objetos é a de ilus-

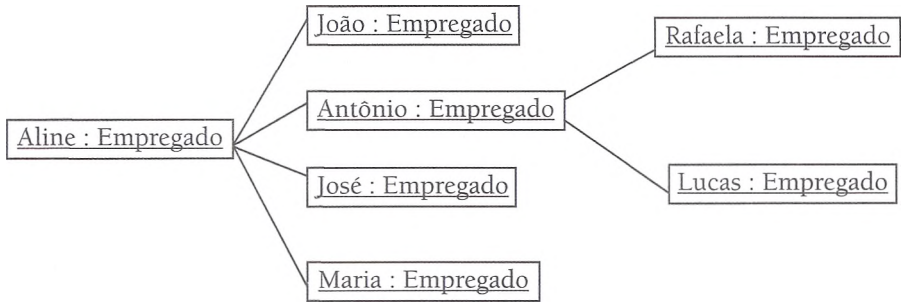


Figura 5-36: Exemplo de diagrama de objetos em que os valores dos atributos não são exibidos.

trar a formação de relacionamentos complexos de um diagrama de classes, como associações reflexivas. Com o objetivo de facilitar a atividade de *validação* (ver Seção 2.1.2), podemos construir diagramas de objetos para ilustrar e esclarecer certos aspectos de um diagrama de classes. Entretanto, fora desse contexto de validação, a descrição do diagrama de objetos ainda é relevante, pois o diagrama de interação (particularmente, o diagrama de comunicação), descrito no Capítulo 7, utiliza a mesma notação que o primeiro.

5.4 Técnicas para identificação de classes

Na Seção 5.2, descrevemos alguns dos elementos do diagrama de classes. No entanto, apenas o conhecimento da notação existente para produção daquele diagrama não é suficiente para a construção do modelo de classes de um *sistema de software orientado a objetos* (SSOO). De fato, uma das tarefas mais importantes e difíceis no desenvolvimento de um SSOO é a identificação das classes necessárias e suficientes para compor esse sistema.

Nesta seção, nosso objetivo é estudar algumas técnicas de identificação de classes. Antes de continuarmos, porém, é importante darmos uma explicação do termo "identificação de classes". Por definição, um SSOO é composto de uma coleção de objetos que colaboram para realizar as tarefas desse sistema. Por outro lado, sabemos que todo objeto de um SSOO pertence a uma classe. Portanto, quando se fala em "identificação das classes", o objetivo na verdade é saber quais objetos irão compor o sistema em questão.

Independentemente da técnica utilizada, a tarefa de identificação de classes se divide em duas atividades. Primeiramente, *classes candidatas* (ou seja, entidades que podem se tornar classes) são identificadas. Depois disso, são aplicados alguns princípios para eliminar classes candidatas desnecessárias. A segunda atividade é, sem dúvida, a mais complicada: identificar *possíveis* classes para um sistema não é complicado; o difícil é eliminar desse conjunto o que não é necessário. Além disso, é importante notar que as atividades para identificação de classes *não* são seqüenciais (uma não começa quando a outra acaba). Ao contrário, os desenvolve-

dores intercalam a realização dessas atividades, identificando novas candidatas e removendo candidatas previamente identificadas. Parafraseando Bertrand Meyer: "Como um jardineiro, o [desenvolvedor] deve, a todo o momento, alimentar as plantas boas e eliminar as ervas daninhas" (Meyer, 1997).

5.4.1 Análise textual de Abbott

Em 1983, Russell Abbott propôs essa técnica de identificação que hoje leva seu nome (Abbott, 1983). Na chamada *Análise Textual de Abbott* (ATA), utilizam-se diversas fontes de informação sobre o sistema: documento de requisitos, modelos do negócio (se existirem; ver Seção 4.6.1), glossários, conhecimento sobre o domínio etc. Para cada um desses documentos, os nomes (substantivos e adjetivos) que aparecem no mesmo são destacados. (São também consideradas locuções equivalentes a substantivos.) Após isso, os sinônimos são removidos (permanecem os nomes mais significativos para o domínio do negócio em questão). Cada termo remanescente se enquadra em uma das situações a seguir:

1. O termo se torna uma classe (ou seja, são classes candidatas).
2. O termo se torna um atributo.
3. O termo não tem relevância alguma com relação aos requisitos do SSOO.

Abbott segue na descrição de sua proposta para aplicá-la também na identificação de *operações* de uma classe e de *associações* entre classes. Para isso, ele sugere que destaquemos os verbos no texto. Verbos com sentido de ação (por exemplo calcular, confirmar, cancelar, comprar, fechar, estimar, depositar, sacar etc.) são operações em potencial. Verbos com sentido de "ter" são agregações ou composições (ver Seção 5.2.2.7) em potencial. Verbos com sentido de "ser" são generalizações (ver Seção 5.2.3) em potencial. Os demais verbos são associações em potencial.

Conforme descrito no parágrafo anterior, uma operação é identificada mediante análise de um verbo que denota ação. A classe na qual essa operação deve ser definida pode ser encontrada através do sujeito da frase em que o verbo é utilizado. Além disso, informações necessárias à realização dessa operação podem normalmente ser encontradas no complemento da frase.

Parte do Texto	Componente	Exemplo
Nome próprio	Objeto	Eduardo Bezerra
Nome simples	Classe	aluno
Verbos de Ação	Operação	registrar
Verbo Ser	Herança	é um
Verbo Ter	Todo-parte	tem um

Uma vantagem da análise de Abbott é que essa abordagem é bastante simples: para cada documento, destacam-se termos para detectar componentes do modelo de classes. No entanto, apesar da facilidade de aplicação dessa técnica, uma desvantagem é que seu resultado (as classes candidatas identificadas) depende de o documento utilizado como fonte ser completo. Além disso, dependendo do estilo que foi utilizado para escrever esse documento, essa técnica pode levar à identificação de diversas classes candidatas que não gerarão classes. Pior que isso, a análise do texto de um documento pode não deixar explícita uma classe importante para o sistema. Isso porque, em linguagem natural, as variações lingüísticas e as formas de expressar uma mesma idéia são bastante numerosas.

Para contornar os problemas na identificação de classes através da análise textual, uma solução é aplicar outra técnica para validar o que foi descoberto inicialmente e para identificar novas classes.

5.4.2 Análise dos casos de uso

Essa técnica é um caso particular da Análise Textual de Abbott, descrita na Seção 5.4.1. A análise de caso de uso é preconizada pelo processo de desenvolvimento denominado RUP (*Rational Unified Process*). Essa técnica é também chamada de *identificação dirigida por casos de uso*. Na análise dos casos de uso, o MCU (ver Capítulo 4) é utilizado como ponto de partida. Conforme vimos no Capítulo 4, um caso de uso corresponde a um comportamento específico do sistema. Em um SSOO, esse comportamento somente pode ser produzido por objetos que compõem o sistema. Em outras palavras, a realização de um caso de uso é responsabilidade de um conjunto de objetos que devem colaborar para produzir o resultado daquele caso de uso. Com base nisso, o modelador que aplica a técnica de análise dos casos de uso tenta identificar as classes necessárias para produzir o comportamento que está documentado na descrição do caso de uso.

A técnica de análise dos casos de uso é aplicada como segue. O modelador estuda a descrição textual de cada caso de uso para identificar classes candidatas. Para cada caso de uso, seu texto (fluxos principal, alternativos e de exceção, pós-condições e precondições etc.) é analisado. Na análise de certo caso de uso, o modelador tenta identificar classes que possam fornecer o comportamento do mesmo. Na medida em que os casos de uso são analisados um a um, as classes do SSOO são identificadas. Quando todos os casos de uso tiverem sido analisados, todas as classes (ou pelo menos a grande maioria delas) terão sido identificadas.

A justificativa para essa técnica é que a existência de uma classe só pode se justificar se ela participar de alguma forma do comportamento externamente

visível do sistema. Dessa forma, o modelo de classes é obtido a partir da descrição dos casos de uso.

De forma resumida, os passos na análise de casos de uso podem ser descritos da seguinte forma:

1. Suplemente as descrições dos casos de uso. Esse passo envolve complementar a descrição dos casos de uso com o objetivo de torná-los completos e facilitar a identificação de todas as classes envolvidas no mesmo.
2. Para cada caso de uso:
   a. Identifique classes a partir do comportamento do caso de uso.
   b. Distribua o comportamento do caso de uso pelas classes identificadas.
3. Para cada classe de análise resultante
   a. Descreva suas responsabilidades.
   b. Descreva atributos e associações.
4. Unifique as classes de análise identificadas em um ou mais diagramas de classes.

Com relação ao passo de identificação das classes a partir do comportamento do caso de uso, há uma categorização de objetos que pode servir como ponto de partida para tal tarefa. Descrevemos essa categorização na Seção 5.4.2.1. Além disso, a *modelagem de interações* na fase de análise se encaixa no contexto da realização dos passos descritos anteriormente. Descrevemos os diagramas de interação no Capítulo 7.

5.4.2.1 Categorização BCE

Uma técnica de identificação que pode ser combinada com a análise de casos de uso é a *Análise de Robustez*. Essa técnica foi proposta por Ivar Jacobson (Jacobson *et al.*, 1992) e posteriormente adotada pelo processo de desenvolvimento denominado ICONIX (Rosenberg & Scott, 2001). De acordo com essa técnica, os objetos que compõem um SSOO podem ser divididos em três categorias: *objetos de fronteira*, *objetos de controle* e *objetos de entidade*. Chamamos essa categorização de BCE (para lembrar os nomes originais das categorias: *boundary*, *control* e *entity*). Essa categorização fornece uma espécie de "arcabouço" que podemos tomar como ponto de partida para a identificação de classes em cada caso de uso de um sistema.

No contexto do diagrama de classes, a UML fornece estereótipos (ver Seção 3.1) predefinidos, tanto textuais, quanto gráficos, para cada categoria BCE. A Figura 5-37 apresenta de forma esquemática os modos de representar objetos em cada uma dessas três categorias.

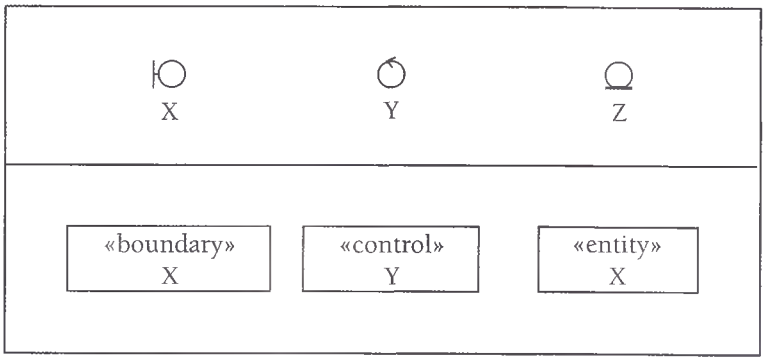


Figura 5-37: Notação da UML para objetos, segunda a categorização BCE.

5.4.2.2 Objetos de fronteira

Objetos de fronteira realizam a comunicação do sistema com atores. Em outras palavras, são objetos de fronteira que permitem ao sistema interagir com seu ambiente. Há três tipos principais de classes de fronteira: as que realizam a interface com o usuário (atores humanos), as que realizam a interface com sistemas externos e as que realizam comunicação com dispositivos atrelados ao sistema.

Para dar um exemplo, considere nosso estudo de caso do SCA (Sistema de Controle Acadêmico). Mais especificamente, analise o caso de uso `Realizar Inscrição` (ver Seção 4.7). Este caso de uso apresenta dois atores, `Aluno` e `Sistema de Faturamento`. Para que o sistema se comunique com esses atores, é necessário que existam objetos para realizar essa comunicação. Por essa razão, criamos as classes `FormulárioInscrição` e `SistemaFaturamento` (consulte a Figura 5.47).

Objetos de fronteira têm tipicamente as seguintes responsabilidades: (1) notificar aos demais objetos os eventos gerados pelo ambiente do sistema e (2) notificar aos atores o resultado de interações entre os demais objetos. Em resumo, objetos de fronteira são especializados em realizar *comunicação com o ambiente* do SSOO. Nomes de classes de fronteira não devem conter verbos nem devem sugerir ações. Em vez disso, esse nome deve servir para lembrar qual é o canal de comunicação com o mundo externo que a classe representa. Exemplos: `FormulárioInscrição`, `LeitoraCartões`, `SistemaFaturamento` etc.

Objetos de fronteira se comunicam com atores e com controladores (ver Seção 5.4.2.3). Normalmente existem somente durante a realização do caso de uso (isso é particularmente verdadeiro para objeto de fronteira que correspondem a interfaces gráficas com o usuário).

Cabe uma observação importante sobre objetos de fronteira: durante a análise da aplicação (ver Seção 2.1.2), o modelador deve abstrair os detalhes dessa

categoria de objetos. Isso significa que, durante a análise, objetos de fronteira devem ser considerados somente como o ponto de contato com o ambiente do sistema. Em particular, os detalhes de *como* será feita essa comunicação com o ambiente (por interface gráfica, por algum protocolo de comunicação etc.) devem ser ignorados na fase de análise. O objetivo nesta fase é identificar as classes e suas responsabilidades em um nível alto de abstração, sem comprometer eventuais soluções técnicas.

5.4.2.3 Objetos de controle

Um objeto de controle é também chamado de *controlador*. Objetos de controle servem como uma ponte de comunicação entre objetos de fronteira e objetos de entidade. São responsáveis por coordenar a execução de alguma funcionalidade específica do sistema. Normalmente (embora não necessariamente), essa parte do sistema corresponde a um caso de uso. Esses objetos decidem o que o sistema deve fazer quando ocorre um evento externo relevante. Eles realizam o controle do processamento; ou seja, servem como "gerentes" dos outros objetos para a realização de um ou mais cenários de um caso de uso. Assim como objetos de fronteira, essa categoria de objetos tem vida curta: em geral, existem somente durante a realização de um caso de uso.

Nomes normalmente utilizados para uma classe de controle devem lembrar o caso de uso que a mesma é responsável por coordenar. Alguns exemplos de nomes de classes de controle são: `GerenciadorContas`, `ControladorInscrição`, `ControladorReservasCarros`, `MarcadorTempo` etc.

Algumas responsabilidades típicas de objetos de controle são: (1) realizar monitorações, a fim de responder a eventos externos ao sistema (gerados por objetos de fronteira); (2) coordenar a realização de um caso de uso por meio do envio de mensagens a objetos de fronteira e a objetos de entidade; (3) criar associações entre alguns objetos de entidade (embora isso possa ser feito pelos próprios objetos de entidade; ver mais adiante); (4) manter valores acumulados ou derivados durante a realização de um caso de uso; (5) manter o estado da realização do caso de uso.

5.4.2.4 Objetos de entidade

Um objeto de entidade representa um conceito encontrado no domínio do problema (ver Seção 2.1.1). Normalmente servem como um repositório para alguma informação manipulada pelo sistema. Mas não somente isso. Nessas classes é que são alocadas as responsabilidades mais importantes do sistema, a saber, as que dizem respeito à *lógica do negócio*. Esses objetos são encontrados durante a *análise de domínio* (ver Seção 2.1.2). É comum existirem várias instâncias de uma mesma classe de entidade coexistindo no

sistema.[3] Por exemplo, em um sistema de venda de produtos, é possível haver milhares de objetos (instâncias) da classe Produto.

Objetos de entidade frequentemente participam de vários casos de uso e têm um ciclo de vida longo (ou seja, armazenam informações persistentes do sistema). Por exemplo, um objeto Aluno pode participar na realização dos casos de uso Realizar Inscrição e Lançar Notas (ver o estudo de caso do SCA). Além disso, uma vez criado, esse objeto pode existir por diversos anos ou mesmo tanto quanto o próprio sistema de que é componente. (No entanto, alguns objetos de entidade podem ter ciclo de vida curto e não ser persistentes.)

Algumas responsabilidades típicas de objetos de entidade são as seguintes: (1) informar valores de seus atributos a objetos requisitantes; (2) realizar cálculos ou impor restrições relativas às *regras do negócio* (ver Seção 4.5.1), normalmente com a colaboração de objetos de entidade associados; (3) criar e destruir objetos-parte (considerando que o objeto de entidade em questão seja um objeto-todo de uma agregação ou composição).

5.4.2.5 Categorização BCE na identificação de classes

A categorização proposta por Jacobson implica que cada objeto é especialista em realizar um dos três tipos de tarefa, a saber: comunicar-se com atores (fronteira), manter as informações (entidade) ou coordenar a realização de um caso de uso (controle).

Pelo exposto nas Seções 5.4.2.2, 5.4.2.3 e 5.4.2.4, podemos concluir que a categorização BCE funciona como uma espécie de "receita de bolo" para identificar objetos participantes da realização de um caso de uso. Em particular, essa técnica preconiza que, para cada caso de uso, as seguintes regras podem ser aplicadas na análise:

1. Adicionar um objeto de fronteira para cada ator participante do caso de uso. Dessa forma as particularidades de comunicação com cada ator do caso de uso ficam encapsuladas no objeto de fronteira correspondente.
2. Adicionar um objeto de controle para o caso de uso. Isso porque um caso de um representa um determinado processo do negócio. O controlador de um caso de uso tem então a atribuição de coordenar a realização desse processo do negócio.

A Figura 5-38 ilustra de forma esquemática o que acontece quando os objetos de um caso de uso de um SSOO são identificados conforme a categorização BCE. O ator se comunica com um objeto de fronteira, que repassa (pelo envio de mensa-

[3] Esses objetos normalmente são armazenados em algum meio persistente. O Capítulo 12 descreve como esses objetos podem ser mapeados e armazenados em um banco de dados relacional.

gens) as ações desse ator para o objeto de controle (controlador). O controlador, por sua vez, coordena a colaboração de um ou mais objetos de entidade para a realização da tarefa desejada pelo ator. Note que, de acordo com o esquema dessa Figura, os objetos de entidade também se comunicam entre si com o envio de mensagem. Note também que a realização de uma tarefa pode envolver o envio de mensagens para outros objetos de fronteira, para que o sistema se comunique com outros atores (por exemplo, outros sistemas) envolvidos no caso de uso em questão. Após a conclusão da tarefa, o objeto de controle repassa o resultado para um objeto de fronteira (que pode ser o mesmo no qual a realização começou, ou outro objeto). Este objeto de fronteira, por sua vez, apresenta o resultado para o ator.

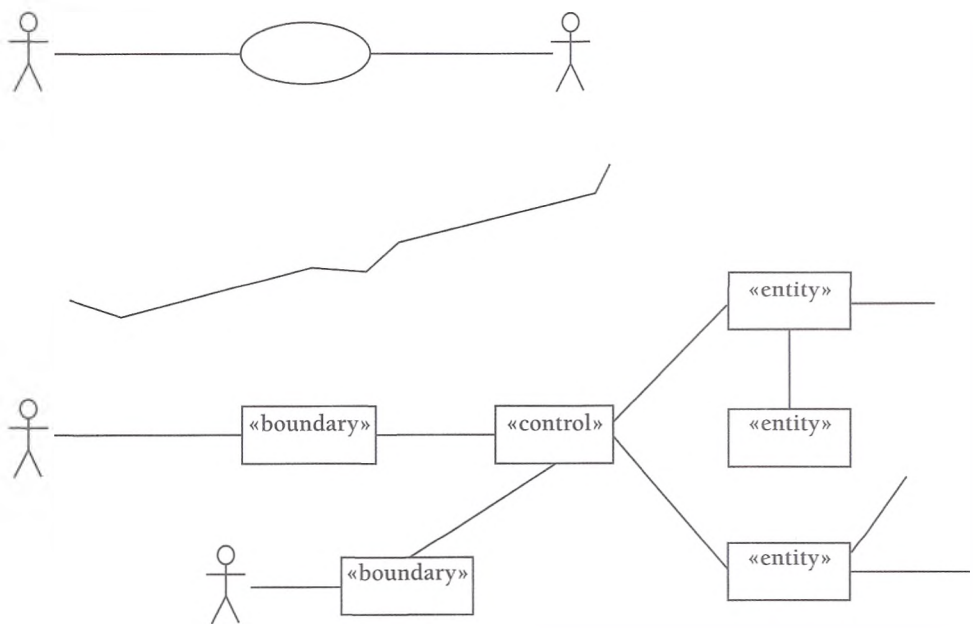


Figura 5-38: A realização de um caso de uso envolve objetos de fronteira, de controle e de entidade.

Classes de entidade, assim como seus atributos, são relativamente fáceis de identificar. Isso porque tipicamente correspondem a conceitos pertencentes ao domínio do problema. Essas classes são identificadas na *análise do domínio*. Já as classes de entidade e de controle são normalmente identificadas na *análise da aplicação*. (Para detalhes acerca da análise de domínio e da análise da aplicação, ver Seção 2.1.2.)

5.4.3 Identificação dirigida a responsabilidades

Na técnica de identificação dirigida a responsabilidades, a ênfase está na identificação de classes a partir de seus *comportamentos* relevantes para o sistema. O esforço do modelador recai sobre a identificação das *responsabilidades* que cada

classe deve ter dentro do sistema. Esse método foi proposto por Rebecca Wirfs-Brock e Brian Wilkerson (Wirfs-Brock e Wilkerson, 1989). Nas palavras desses autores: "O método dirigido a responsabilidades enfatiza o encapsulamento da estrutura e do comportamento dos objetos." Ou seja, essa metodologia utiliza o princípio do encapsulamento, descrito na Seção 1.2.3.1: a ênfase está na identificação das responsabilidades de uma classe que são úteis externamente à mesma. Os detalhes internos à classe (*como* ela faz para cumprir suas responsabilidades) devem ser abstraídos.

Em um SSOO, os objetos encapsulam tanto dados quanto comportamento. O comportamento de um objeto é definido de tal forma que ele possa cumprir com suas *responsabilidades*. Uma responsabilidade de um objeto é uma obrigação que este tem para com o sistema no qual ele está inserido. Graças às suas responsabilidades, um objeto colabora com outros objetos para que os objetivos do sistema sejam alcançados. Na prática, uma responsabilidade é algo que um objeto *conhece* ou *faz (sozinho ou sendo ajudado por outro(s) objeto(s))*. Como exemplo do conceito de responsabilidade, considere um objeto Cliente. Esse objeto provavelmente *conhece* seu nome, seu endereço, seu telefone etc. Considere um objeto Pedido; esse objeto conhece sua data de realização. Ele também deve fazer o cálculo do seu valor total.

Um objeto cumpre com suas responsabilidades a partir das informações que ele possui ou das informações que ele pode derivar de colaborações com outros objetos.

Em alguns casos, um objeto tem uma responsabilidade com a qual ele não pode cumprir sozinho. Nesses casos, o objeto deve requisitar *colaborações* de outros objetos do sistema para cumprir com a sua responsabilidade. Por exemplo, quando a impressão da fatura de um pedido é requisitada em um sistema de vendas, vários objetos precisam colaborar. O diagrama de classes da Figura 5-39 ilustra as classes de alguns objetos que estão envolvidos nessa colaboração. Um objeto `Pedido` pode ter a responsabilidade de fornecer o seu valor total; um objeto `Cliente` fornece seu nome; cada `ItemPedido` informa a quantidade do produto correspondente e o valor de seu subtotal; os objetos `Produto` também colaboram fornecendo seu nome e preço unitário.

De uma forma geral, cada objeto tem um conjunto de responsabilidades dentro de um SSOO. Esse objeto tem como cumprir sozinho algumas dessas responsabilidades. Já para outras responsabilidades, o objeto necessita da colaboração de outros objetos.[4] Estes últimos são seus *colaboradores*. A Figura 5-40 ilustra a correspondência entre classes (de objetos), responsabilidades e colaborações.

---

4. É através da *troca de mensagens* de um objeto a outro que essas colaborações são acionadas para que as funcionalidades do sistema estejam disponíveis externamente. A modelagem de mensagens entre objetos é descrita no Capítulo 7.

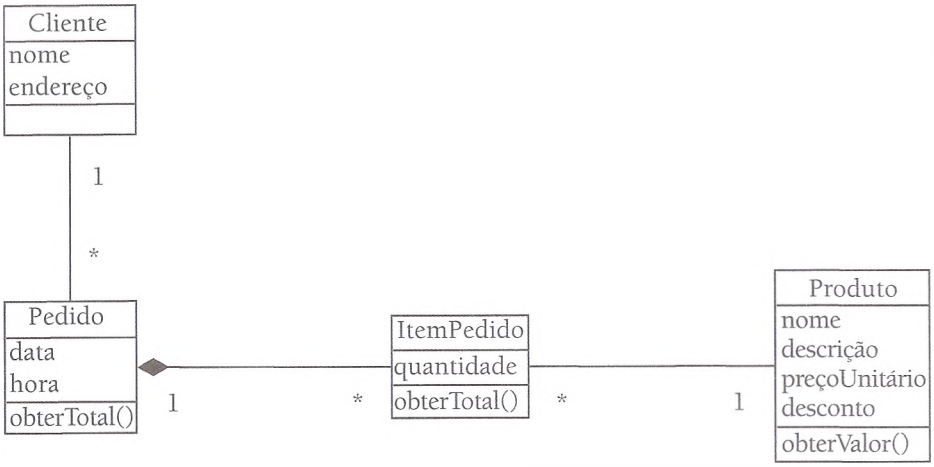


Figura 5-39: Objetos colaboram entre si para cumprir com suas responsabilidades.

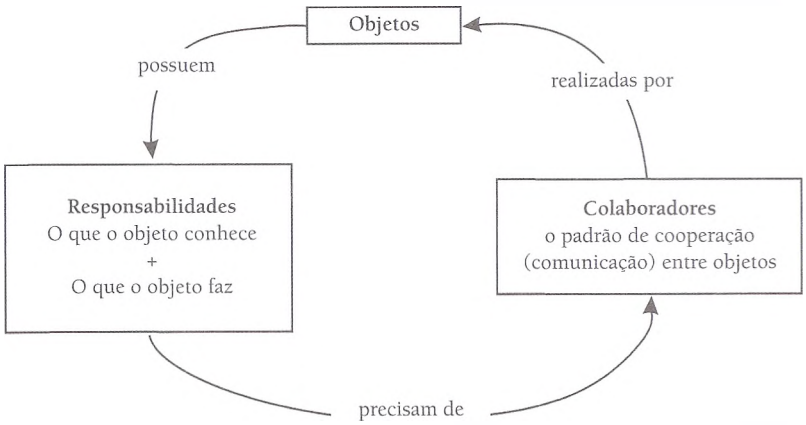


Figura 5-40: Relação entre objetos, responsabilidades e colaboradores.

A identificação de classes dirigida a responsabilidades normalmente utiliza uma técnica que permite a participação de especialistas do domínio e analistas. Essa técnica, a *modelagem CRC*, é descrita na Seção 5.4.3.1.

5.4.3.1 Modelagem CRC

A *modelagem CRC* (sigla para *Classes, Responsabilidades e Colaboradores*) foi proposta em 1989 por Kent Beck[5] e Ward Cunningham (Kent e Cunningham, 1989). A princípio, essa técnica de modelagem foi proposta como uma forma de

5. Esse autor também é o principal expoente de um processo de desenvolvimento que está se tornando bastante famoso, o XP (sigla para *Extreme Programming*).

ensinar a iniciantes o paradigma da orientação a objetos. Contudo, a sua simplicidade de notação a tornou particularmente interessante para ser utilizada na identificação de classes.

Essa técnica se baseia fortemente no paradigma da orientação a objetos, em que objetos colaboram uns com os outros para que uma tarefa seja realizada. Ela é eficaz quando profissionais que não têm tanta experiência com o paradigma da orientação a objetos (o que normalmente é o caso para especialistas do domínio) estão envolvidos na identificação de classes. A idéia básica dessa técnica é a de que as responsabilidades atribuídas a um SSOO devem, em última análise, ser atribuídas aos objetos componentes do mesmo. Sendo assim, podemos partir das responsabilidades que atribuímos ao sistema e, para cada uma delas, identificar objeto para assumir com as mesmas (ver Figura 5-41).

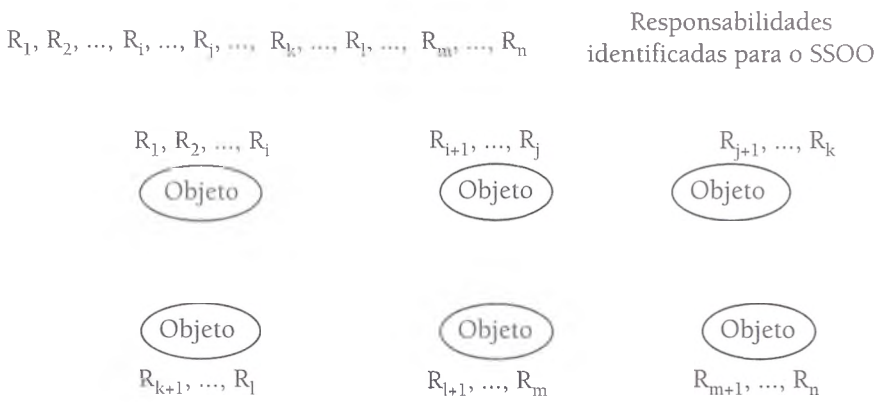


Figura 5-41: As responsabilidades de um sistema de software orientado a objeto devem ser alocadas aos objetos componentes do mesmo.

Essa técnica é útil na identificação de novas classes e na validação das classes identificadas mediante outras técnicas (como, por exemplo, a análise de casos de uso). Na modelagem CRC, especialistas do domínio e desenvolvedores trabalham em conjunto para identificar objetos, suas responsabilidades e colaboradores. Para aplicar essa técnica, esses profissionais se reúnem em uma sala, onde tem início uma *sessão CRC*.

Uma sessão CRC é uma reunião que envolve cerca de seis pessoas. Entre os participantes estão especialistas de domínio, projetistas, analistas e o moderador da sessão.

A cada pessoa é entregue um cartão de papel como o da Tabela 5-4.[6] Esse cartão é denominado *cartão CRC* e mede aproximadamente 10cm x 15cm. Cada cartão corresponde a uma das classes do SSOO.

[6] De preferência, cada participante deve ficar com único cartão. Isso faz com que o participante se identifique com a classe correspondente.

Tabela 5-5: Formato típico de um cartão CRC

Nome da classe	
Responsabilidades	Colaboradores
1ª responsabilidade	1º colaborador
2ª responsabilidade	2º colaborador
...	..
n-ésima responsabilidade	*m*-ésimo colaborador

A Tabela 5-6 ilustra um exemplo de cartão CRC para a classe ContaBancária. A coluna de colaboradores desse cartão foi definida em função das responsabilidades alocadas à classe.

Tabela 5-6: Cartão CRC para a classe Conta Bancária

ContaBancária	
Responsabilidades	Colaboradores
1. Conhecer o seu cliente	Cliente
2. Conhecer o seu número	Transação
3. Conhecer o seu saldo	
4. Manter um histórico de transações	
5. Aceitar saques e depósitos	

Uma vez distribuídos os cartões pelos participantes, um conjunto de cenários (ver Seção 4.1.1.4) de determinado caso de uso é selecionado. Então, para cada cenário desse conjunto, uma sessão CRC é iniciada. (Se o caso de uso não for tão complexo, ele pode ser analisado em uma única sessão.)

Note que um cartão CRC é dividido em várias partes, conforme podemos ver na Tabela 5-5. Na parte superior do cartão, aparece o nome de uma classe. A parte inferior do cartão é dividida em duas colunas. Na coluna da esquerda, o indivíduo ao qual foi entregue o cartão deve listar as responsabilidades da classe. Finalmente, na coluna da direita, o indivíduo deve listar os nomes de outras classes que colaboram com a classe em questão para que ela cumpra suas responsabilidades.

É normal já existirem algumas classes candidatas já identificadas para determinado cenário (obtidas com a aplicação de outras técnicas de identificação).[7]

[7] Classes candidatas iniciais também podem ser encontradas em entrevistas com o usuário, no documento de requisitos, no modelo de regras do negócio etc.

A sessão CRC começa com algum dos participantes simulando o ator primário que dispara a realização do caso de uso. À medida que esse participante simula a interação do ator com o sistema, os demais participantes encenam a colaboração entre objetos que ocorre internamente ao sistema, para que o objetivo do ator seja alcançado. Graças a essa encenação desempenhada pelos participantes da sessão CRC, as classes, responsabilidades e colaborações são identificadas.

Na modelagem CRC, as diversas responsabilidades de um SSOO são atribuídas a classes. Essa técnica permite *antropomorfizar* o conceito de classe. Ou seja, objetos são verdadeiramente interpretados como entidades ativas que encapsulam comportamento.

Durante uma sessão CRC, para cada responsabilidade atribuída a uma classe, o seu proprietário (o indivíduo ao qual foi alocado o cartão correspondente) deve questionar se tal classe é capaz de cumprir a responsabilidade sozinha. Se ela precisar de ajuda, essa ajuda é dada por um colaborador. Os participantes da sessão, então, decidem que outra classe pode fornecer tal ajuda. Se essa classe existir, ela recebe uma nova responsabilidade, necessária para que ela forneça ajuda. Do contrário, um novo cartão (ou seja, uma nova classe) é criado para cumprir com tal responsabilidade de ajuda.

Uma classe pode participar em mais de um caso de uso. À medida que os cenários de casos de uso são encenados, as responsabilidades (e os colaboradores) dessa classe vão se acumulando. Ou seja, as responsabilidades e os colaboradores necessários para a realização dos cenários aparecem de acordo com o desenrolar da sessão CRC. Além disso, durante uma sessão CRC, uma ou mais das seguintes ações podem ser necessárias:

- Adicionar uma responsabilidade identificada a uma classe já existente.
- Criar uma classe para realizar uma responsabilidade identificada.
- Mover responsabilidades quando uma classe se tornar muito complexa.

5.4.3.2 Dicas para atribuição de responsabilidades

Uma sessão CRC é uma reunião bastante dinâmica. Em cada encenação, responsabilidades podem passar de um objeto para outro, novos objetos podem ser criados para assumir outras responsabilidades, objetos preexistentes podem não ser mais necessários etc. A seguir, descrevemos algumas dicas que podem ser usadas durante uma sessão CRC para atribuir responsabilidades a classes. Essas mesmas dicas devem ser seguidas durante a modelagem de interações (ver Capítulo 7).

Associe responsabilidades com base na especialidade da classe. Dependendo da especialidade (fronteira, controle ou entidade) da classe, há um conjunto típico de responsabilidades com as quais ela deve cumprir (ver Seção 5.4.2.1). O modelador pode tomar esse conjunto como base para atribuir responsabilidades às classes do sistema.

Distribua a inteligência do sistema. A inteligência do sistema deve ser uniformemente distribuída. Isso quer dizer que o conjunto de responsabilidades do sistema deve ser distribuído o mais uniformemente possível pelas classes. Com efeito, uma classe não deve ser sobrecarregada com responsabilidades demais. Se isso começar a acontecer com uma classe, considere a criação de outras classes para assumirem algumas responsabilidades que pertencem à classe sobrecarregada. Uma quantidade pequena de classes complexas demais significa que as mesmas encapsulam mais conhecimento (inteligência) do que o desejado e são menos reutilizáveis. Por outro lado, uma quantidade maior de classes mais simples significa que cada classe encapsula uma fração menor da inteligência do sistema e, assim, são mais reutilizáveis.

Como um exemplo, considere que a um objeto Pedido foi atribuída a responsabilidade de conhecer o seu valor total. Para isso, é necessário que todos os subtotais dos itens de pedido sejam calculados. Cada item de pedido pode ficar responsável por calcular o seu total e informar ao objeto pedido para que ele faça a totalização. Por sua vez, o item de pedido pode pedir ajuda ao objeto Produto para que este calcule o seu valor unitário (necessário ao cálculo do subtotal). A situação pode ser resumida da seguinte forma:

1. Produto sabe o seu valor total.
2. ItemPedido sabe o seu subtotal.
3. Produto sabe o seu valor.

Nessa situação, cada classe fica com uma parte da responsabilidade relacionada à tarefa de computar o valor total de um pedido. A inteligência necessária para o cálculo desse valor é distribuída pelo sistema.

Agrupe as responsabilidades conceitualmente relacionadas. Responsabilidades conceitualmente relacionadas devem ser mantidas em uma única classe. Por exemplo, as informações (responsabilidades *de conhecer*) pertinentes a um cliente devem estar definidas em uma classe Cliente, e não espalhadas por diversas classes. Da mesma forma, é mais adequado criar as classes chamadas Pedido e Fatura em vez de uma única classe para manter informações sobre os dois conceitos. Essa dica está diretamente relacionada com o conceito de *coesão* de uma classe, que detalhamos na Seção 7.5.2.

Evitar responsabilidades redundantes. Responsabilidades redundantes não devem ser criadas. Se duas ou mais classes precisam de alguma informação, analise as seguintes decisões alternativas: criar uma nova classe ou escolher uma das classes preexistentes para manter a informação. Em ambos os casos, a informação fica em um único lugar, e os outros objetos devem enviar mensagens para o objeto que possui tal informação para obtê-la.

A decisão de criar uma nova classe ou utilizar uma classe preexistente para manter a informação deve ser tomada com cuidado. É recomendável que o modelador tente sempre reutilizar uma classe preexistente para atribuir uma responsabilidade. No entanto, deve-se observar outra dica anterior: não se deve adicionar uma responsabilidade que não esteja relacionada ao propósito da classe; nesse caso, é melhor criar uma nova classe.

5.4.4 Padrões de análise

Após produzir diversos modelos para um mesmo domínio de problema, é natural que um modelador comece a identificar características comuns entre esses modelos. Em particular, um mesmo conjunto de classes ou colaboração entre objetos costuma recorrer, com algumas pequenas diferenças, em todos os sistemas desenvolvidos para o domínio do problema em questão. Por exemplo, quantos modelos de classes já foram construídos que possuem os conceitos Cliente, Produto, Fornecedor, Departamento etc? Quantos outros já foram construídos que possuíam os conceitos Ordem de Compra, Ordem de Venda, Orçamento, Contrato etc.?

A identificação de características comuns é um processo natural e acontece em conseqüência do ganho de experiência do modelador em determinado domínio de problema. Ao reconhecer processos e estruturas comuns em um domínio de problema, o modelador pode descrever (catalogar) a parte essencial dos mesmos, dando origem a um *padrão de software*. Quando o problema correspondente a um padrão de software acontecer, um modelador pode utilizar a essência da solução descrita pelo padrão (descrição essa possivelmente catalogada por outro modelador). A aplicação desse processo permite que o desenvolvimento de determinado aspecto de um SSOO seja feito de forma mais rápida e menos suscetível a erros. Isso porque a reutilização de uma mesma solução já comprovada anteriormente livra o modelador da tarefa de repensar (ou reinventar) tal solução.

Um padrão de software pode, então, ser definido como uma descrição essencial de um problema recorrente no desenvolvimento de software. Padrões de *software vêm sendo estudados há anos e são atualmente utilizados em diversas* atividades do desenvolvimento de um sistema, inclusive na análise. Padrões de *software utilizados na análise são chamados padrões de análise.* Um padrão de

análise normalmente é composto, entre outras partes, de um fragmento de diagrama de classes que pode ser customizado para uma situação de modelagem em particular.

Pois bem, a técnica de identificação de classes pela utilização de padrões de análise consiste em identificar padrões que podem ser aplicados à modelagem do SSOO em questão. Sendo assim, o que ocorre não é uma identificação de classes propriamente, mas, sim, a identificação de problemas cujas soluções podem ser encontradas em padrões de análise.

Nos últimos anos, padrões de análise vêm sendo extensivamente estudados e catalogados. Está fora do escopo deste livro um estudo detalhado sobre padrões de análise. No entanto, com o objetivo de dar ao leitor um entendimento em alto nível sobre o assunto, nas próximas seções descrevemos alguns padrões de análise existentes.

5.4.4.1 O padrão Party

O padrão Party é um dos padrões documentados em (Hay, 1996). Também foi descrito em (Fowler, 1997). Esse padrão é normalmente utilizado para representar componentes de uma organização e os relacionamentos entre eles. Esse padrão trabalha com o conceito de *parte* (tradução para *party*). Uma parte é uma generalização de uma pessoa ou de uma organização de interesse para a aplicação sendo modelada. A Figura 5-42 apresenta a estrutura de classes correspondente ao padrão Party.

Na Figura 5-42, a classe Parte possui uma auto-associação que representa relações de subordinação: entre pessoas e organizações, entre pessoas (por

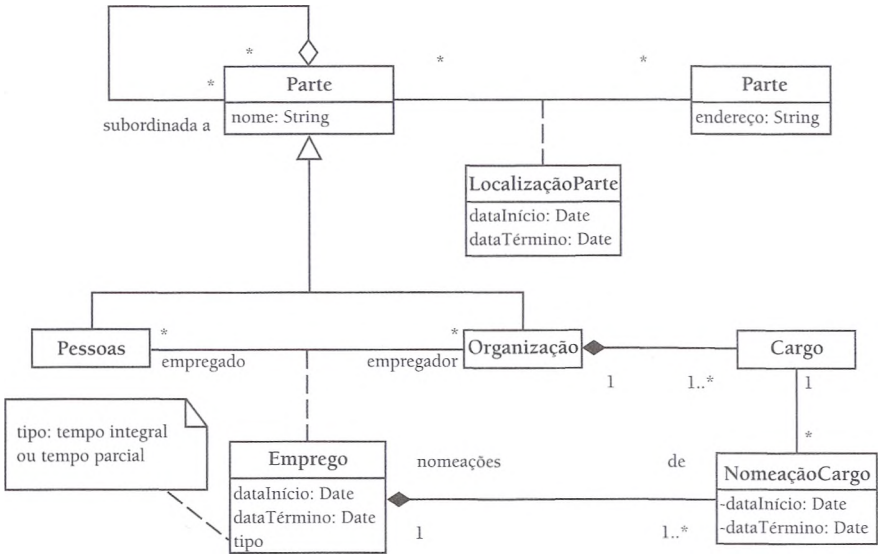


Figura 5-42: O padrão Party.

exemplo, quais as pessoas que supervisionam as outras), ou entre organizações (por exemplo, que organizações são subsidiárias de outras).

Embora não seja exibido na Figura 5-42, tanto `Pessoa` quanto `Organização` podem ter quaisquer atributos que sejam necessários a uma aplicação particular do padrão Party. Note também a existência da classe `Emprego`, que representa associações entre empregados (pessoas) e empregadores (organizações). Essa mesma classe `Emprego` está associada à classe `NomeaçãoCargo`, e essa associação permite identificar que empregados assumiram determinados cargos em quais organizações e em que períodos.

5.4.4.2 O padrão Metamodel

Considere um conjunto de itens quaisquer. Considere, ainda, que cada item possui várias propriedades. Um caso particular dessa situação é o caso em que uma classe possui um grupo (fixo) de atributos, mas, no caso geral, as propriedades de cada item podem ser diferentes das propriedades dos outros itens do conjunto.

Por exemplo, analise os seguintes itens (neste caso, equipamentos): `Monitor`, `Modem`, `Teclado` e `Processador`. Obviamente, as propriedades desses produtos não são as mesmas. É desejável criar no modelo um único conceito para representar Produto, sem deixar de modelar as propriedades particulares de cada tipo de produto. Uma possível solução para esse problema é criar uma classe denominada `Produto` e criar diversas subclasses, uma para cada tipo de produto existente. Dessa forma, cada subclasse representaria as propriedades particulares do tipo de produto correspondente. Uma desvantagem dessa solução é que novos produtos estão sempre aparecendo e outros desaparecendo, o que levaria a uma freqüente modificação na estrutura de modelagem.

Para complicar mais ainda, podem surgir situações em que as propriedades de um mesmo item podem variar com o tempo (ou seja, novas propriedades podem ser adicionadas, ou propriedades existentes podem ser removidas). Se a solução descrita no parágrafo anterior fosse utilizada, haveria muitas mudanças no esquema do banco de dados, o que muito provavelmente acarretaria mudanças nas aplicações que utilizassem esse esquema.

O padrão Metamodel permite a "modificação" da estrutura de um modelo de objetos sem que o esquema desses objetos seja realmente modificado. O que acontece é que o esquema de classes representa um *metamodelo*. Esse padrão é apresentado na Figura 5-43.

Vamos agora descrever o funcionamento do padrão Metamodel. Para isso, considere os equipamentos de computador citados no exemplo anterior. Considere ainda que *`dimensões`*, *`velocidade de transferência`*, *`quantidade de teclas`* e *`velocidade`* são atributos de `Monitor`, `Modem`, `Teclado` e `Processador`, respectivamente. Através do padrão Metamodel, esses atributos são representados como

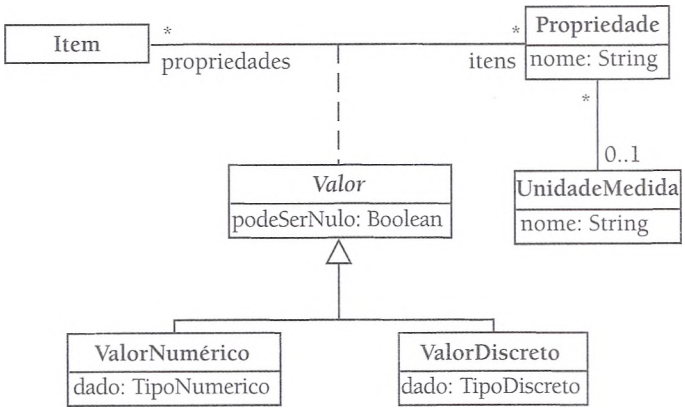


Figura 5-43: O padrão Metamodel.

instâncias da classe `Propriedade`, cada item é representado como uma instância da classe Item. Finalmente, a classe valor é utilizada para representar o valor de uma propriedade de determinado item.

Note que propriedades podem ser adicionadas (ou removidas) do metamodelo cuja base está no padrão Metamodel. Por exemplo, para adicionar o atributo *formato* ao item `Monitor` (para indicar se o monitor possui tela plana ou não), basta adicionar uma nova linha à tabela correspondente ao mapeamento da classe Propriedade. Note também que é responsabilidade da aplicação apresentar os dados como se eles fossem atributos da instância de Item. Pode haver inclusive uma aplicação que permita que seus usuários adicionem (removam) atributos.

5.4.5 Outras técnicas de identificação

Além das técnicas de identificação descritas nas seções anteriores, diversas outras abordagens podem ser aplicadas com o objetivo de identificar as classes de um SSOO. Um exemplo é a utilização de uma taxonomia de conceitos:

- *Conceitos concretos*, como edifícios, carros, salas de aula etc.
- *Papéis* desempenhados por seres humanos, como professores, alunos, empregados, clientes etc.
- *Eventos*, ou seja, ocorrências em uma data e em uma hora particulares, como reuniões, pedidos, aterrisagens, aulas etc.
- *Lugares*, ou seja, áreas reservadas para pessoas ou coisas, como escritórios, filiais, locais de pouso, salas de aula etc.
- *Organizações*, ou seja, coleções de pessoas ou de recursos, como departamentos, projetos, campanhas, turmas etc.
- *Conceitos abstratos*, isto é, princípios ou idéias não tangíveis, como reservas, vendas, inscrições etc.

Além de estudar taxonomias de conceitos, é uma boa abordagem analisar também as funcionalidades e a documentação de sistemas similares no mesmo domínio do problema. O estudo do vocabulário dos especialistas do domínio é também uma boa fonte para identificação de classes.

5.4.6 Discussão

Nas seções anteriores, descrevemos diversas técnicas para identificação de classes. No entanto, uma questão que aflige diversos modeladores (tanto iniciantes, quanto experientes) é a seguinte: é possível aplicar as técnicas de modelagem de dados para identificação de classes? Talvez a principal fonte para esta dúvida seja a semelhança de notação que existe entre o diagrama de entidade e relacionamentos (conhecida ferramenta de modelagem de dados) e o diagrama de classes. Alistair Cockburn (Cockburn, 1996) descreve uma experiência que envolveu dois grupos de profissionais, um de modelagem de dados e o outro de modelagem orientada a objetos. Nessa experiência, foi requisitado aos dois grupos que elaborassem um modelo de domínio para uma dada situação. Nessa experiência, Cockburn constatou que os dois grupos produziram modelos similares, *sob o aspecto estrutural*. No entanto, quando comparado o *aspecto funcional* (os profissionais de modelagem de dados utilizaram o DFD na experiência), os resultados foram bastante diferentes.

De fato, a questão levantada no parágrafo anterior é das mais complexas, para a qual há respostas conflitantes na literatura. Posto isso, o que exponho nas próximas linhas é minha opinião sobre o assunto. A diferença mais significante entre um modelo de dados e um modelo de objetos é o fato de este último representar o *comportamento* dos objetos, além de seus dados.

Em minha experiência prática em modelagem orientada a objetos, constato que as entidades que seriam identificadas com a modelagem de dados freqüentemente são identificadas (sob a forma de classes) com a modelagem orientada a objetos. Isso porque ambas as abordagens têm princípios semelhantes em relação ao que consideram um bom modelo do ponto de vista estrutural. Mas outra constatação minha é que objetos do domínio que têm um viés mais comportamental dificilmente são identificados com a técnica de modelagem conceitual de dados. Isso se justifica pelo simples fato de essa técnica priorizar o aspecto dos dados relacionados a uma dada entidade do mundo real. Além disso, um modelo conceitual de dados representa a estrutura estática, enquanto modelos de classes representam uma fotografia (estática) das estruturas (relacionamentos) que são criadas durante o comportamento dinâmico do sistema. Além disso, diferentemente do modelo conceitual de dados, algumas entidades representadas em um modelo de classes

são transientes (ou seja, não "vivem" além de uma sessão do sistema, armazenadas em um SGBD, por exemplo).

Não se trata de afirmar que a modelagem conceitual de dados seja uma técnica ruim; o fato é que ela deixa para outras técnicas (por exemplo, a decomposição funcional com a ferramenta de Diagrama de Fluxos de Dados) a responsabilidade de considerar o aspecto funcional do sistema que está sendo modelado. Em minha opinião, não há nada de errado em aplicar as técnicas de modelagem conceitual de dados para construir o aspecto estrutural do modelo de classes. No entanto, não se pode pensar que chegamos ao final da tarefa quando tivermos feito isso. É importante considerar também o aspecto comportamental desse modelo. E isso deve ser feito com a aplicação de técnicas de identificação que levem em conta esse aspecto estrutural, algumas das quais foram descritas nesta seção (análise textual de Abbott, análise de casos de uso, modelagem CRC).

Modelos de classes são perigosamente similares a modelos de dados. A maioria dos princípios que resultam em um bom modelo de dados também resultam em um bom modelo de classes. O perigo é justamente desenvolver um modelo de classes orientado a dados, em vez de orientado a responsabilidades.

Outro ponto importante: é recomendável que o modelador aplique duas ou mais técnicas de identificação durante a identificação de classes. O objetivo disso é fazer com que a aplicação de uma técnica valide a aplicação de outra técnica. A utilização de mais de uma técnica também permite descobrir classes que seriam ignoradas se apenas uma técnica fossem aplicada. Além disso, é fundamental a participação de um ou mais especialistas do domínio durante essa atividade. É muito pouco provável que um analista tenha conhecimento equiparável a uma especialista no domínio de conhecimento deste último.

Uma primeira questão que podemos analisar é como conciliar as técnicas de modelagem CRC com a análise textual de Abbott. Ou seja, será que essas duas técnicas podem ser aplicadas em conjunto para identificar as classes de análise? No meu entender, a resposta é positiva. Isso pode ser feito em duas etapas. Em uma primeira etapa, podemos identificar as classes mais óbvias através da ATA (podemos utilizar o documento de especificação de requisitos, descrições de casos de uso, manuais, enfim, quaisquer fontes de informação textual). Note que na técnica ATA, os verbos que indicam ação podem ser interpretados como responsabilidades em potencial. As classes assim identificadas servem de base para a segunda etapa, quando ocorre a aplicação da modelagem CRC. No início de uma sessão CRC, cada indivíduo participante deve ser responsável por ao me-

nos uma classe identificada pela aplicação da ATA. As vantagens dessa abordagem são duas. Em primeiro lugar, as classes identificadas através da ATA são validadas pela aplicação da técnica de modelagem CRC. Em segundo lugar, a sessão CRC começa com um conjunto de classes (e possivelmente responsabilidades) identificadas. Por fim, note que as técnicas de análise de casos de uso e modelagem CRC também podem ser combinadas, visto que a primeira é um caso especial da técnica ATA.

O mesmo raciocínio que empregamos no parágrafo anterior para argumentar sobre a possibilidade de aplicação da ATA juntamente com a modelagem CRC pode ser utilizado em relação a esta última e à análise de casos de uso. Em particular, as responsabilidades de um sistema (que a modelagem CRC considera como ponto de partida) podem ser vistas como os casos de uso identificados para o mesmo. Portanto, outra estratégia válida para identificação de classes é aplicar a análise de casos de uso e, subseqüentemente, validar os resultados com a aplicação da modelagem CRC.

Ainda sobre a conciliação de técnicas de identificação, até aqui tenho descrito formas em que a modelagem CRC é aplicada como forma de validação de outra técnica. Isso não quer dizer que essa técnica seja menos importante que as outras. Na verdade, é justo comentar que há domínios em que a modelagem CRC é mais adequada e mais aplicável que as outras. Por exemplo, na identificação de objetos para um jogo, para um sistema de automação, ou para um sistema de tempo real, essa técnica consegue de forma bastante mais natural capturar todas as responsabilidades do sistema na forma de objetos.

É importante também notar que as técnicas de identificação de classes descritas na Seção 5.3 normalmente são aplicadas de um modo *iterativo*. Iterativo porque é virtualmente impossível para um modelador construir um modelo perfeito da primeira vez. Em vez disso, o modelo de classes adequado para um SSOO é construído por refinamentos sucessivos. Cada refinamento aperfeiçoa a versão anterior do modelo.

Por último, é relevante fazer um comentário acerca da diferença conceitual que há entre as classes que definimos durante a modelagem de classes do domínio e durante a modelagem de classes da aplicação. Em um SSOO, é razoável categorizar seus objetos em dois tipos: *objetos do domínio* e *objetos da aplicação*. Com relação à identificação de classes, quando estamos realizando a modelagem de classe de domínio, normalmente nós *descobrimos* classes, no sentido de que as classes que definimos são abstrações de entidades reais existentes no domínio. Por outro lado, quando estamos na modelagem de classes da aplicação, normalmente *inventamos* classes. Isso porque as classes que definimos nessa atividade *não* têm correspondente no domínio do problema (esse é normalmente o caso das classes de controle e de fronteira).

5.5 Construção do modelo de classes

Após a identificação de classes e atribuição de responsabilidades, o modelador deve verificar a consistência entre as classes para eliminar incoerências e redundâncias. Como dica, o modelador deve estar apto a declarar as razões de existência de cada classe identificada e, para cada classe, argumentar sobre a existência de cada uma de suas responsabilidades e de cada um de seus colaboradores. Do contrário, o modelador deve reconsiderar se a classe é verdadeiramente necessária.

Em seguida, os analistas devem começar a definir o mapeamento das responsabilidades e dos colaboradores de cada classe para os elementos do diagrama de classes. Esse mapeamento gera um diagrama de classes que apresenta uma estrutura estática relativa a todas as classes que foram identificadas anteriormente como participantes da realização de um ou mais casos de uso. A seguir, descrevemos o mapeamento de responsabilidades e colaboradores, levando em consideração os diversos elementos do diagrama de classes apresentados na Seção 5.2.

5.5.1 Definição de propriedades

Uma propriedade é um nome genérico que se aplica aos membros de uma classe, tanto atributos, quanto operações. Uma questão importante é acerca de como podemos mapear (transformar) responsabilidades atribuídas a uma classe para propriedades da mesma.

Com relação às operações, elas correspondem a um modo mais detalhado de explicitar as responsabilidades *de fazer* de uma classe. Uma operação também pode ser vista como uma pequena contribuição da classe para uma tarefa mais complexa representada por um caso de uso. Algumas operações de uma classe podem ser identificadas facilmente. No entanto, uma definição mais detalhada e completa das operações de uma classe só pode ser feita após a construção dos *diagramas de interação*. Esses diagramas representam detalhes sobre a ordem das *colaborações* entre os objetos que participam da realização de um caso de uso. A descrição dos diagramas de interação e sua construção está no Capítulo 7. Portanto, inicialmente, devemos nos preocupar com a descrição precisa das responsabilidades de cada classe, sem perdermos muito tempo com detalhes sobre sua transformação em uma ou mais operações; esses detalhes são definidos na modelagem de interações. Dito isso, no restante desta seção, está o mapeamento de responsabilidades para atributos; enquanto o aspecto do detalhamento de operações está nos capítulos seguintes.

Normalmente uma responsabilidade *de conhecer* é mapeada para atributos ou para associações. Essa tarefa de mapeamento é relativamente fácil, uma vez que estejam corretamente identificadas as responsabilidades de fazer e os co-

laboradores da classe (ver Seção 5.4.3). Após mapear responsabilidades para atributos, o modelador deve verificar se cada atributo possui as seguintes propriedades:

1. **Guarda um valor atômico:** normalmente um atributo armazena um valor atômico como quantidades, quantias, nomes de coisas etc. Exemplos de valores atômicos: R$ 100.00, 5, 2.3, "João da Silva".
2. **Não contém estrutura interna:** esta característica é uma conseqüência da característica anterior. Se uma responsabilidade *de conhecer* possui uma estrutura interna relevante para o sistema e se pertence ao domínio do negócio, considere seriamente modelá-la como uma classe. Exceções são atributos que correspondem a seqüências de texto (string) ou a datas, que são mais adequadamente modelados como atributos (mesmo que no modelo de classes de especificação esses atributos sejam transformados em classes).
3. **Aplica-se a *todos* os objetos da classe:** cada atributo de uma classe deve fazer sentido para todo e qualquer objeto dessa classe. Por exemplo, é incoerente ter o atributo salário em uma classe Pessoa (pois nem toda pessoa tem um salário).

É comum ao iniciante em modelagem de classes mapear uma responsabilidade *de conhecer* como atributos, quando uma melhor modelagem seria mapeá-la como uma associação. Por exemplo, considere a responsabilidade "conhecer o seu cliente" atribuída à classe `Pedido`. Considere também o mapeamento dessa responsabilidade para o atributo `nomeCliente` na classe `Pedido`. Nesse caso, é mais adequado criar uma associação entre `Pedido` e `Cliente`, sendo que esta última possui o atributo nome. De forma geral, se duas classes estão associadas, não há necessidade de criar um atributo em uma classe para fazer referência a informações da outra.[8] A Figura 5-44 ilustra um exemplo dessa confusão entre atributos e associações.

Outro erro comum para iniciantes em modelagem orientada a objetos é a utilização de *atributos de identificação*; por exemplo, em uma classe `Cliente`, definir `idCliente` como um atributo. Embora haja a necessidade de identificar unicamente cada objeto de uma classe, o modelo de classes de domínio não é o lugar para definir esse tipo de atributo. Nesse modelo, somente informações que existem no domínio do negócio devem ser definidas, e as características de implementação devem ser abstraídas. Isso não quer dizer que atributos que corres-

[8] Talvez esse erro comum esteja associado à correlação que as pessoas fazem com o modelo relacional, no qual existe o conceito de chave estrangeira (ver Seção 12.2). No entanto, esse é um conceito que não deve ser utilizado no modelo de classes.

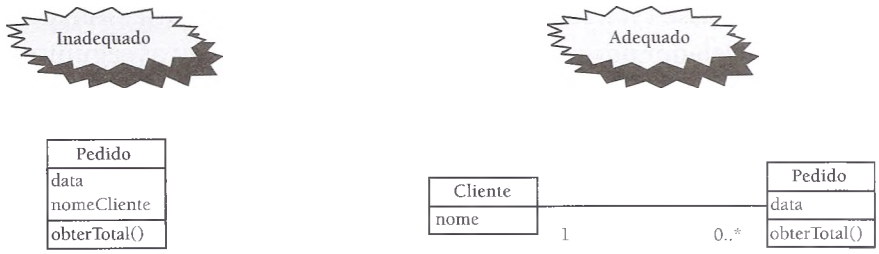


Figura 5-44: Confusão entre atributos e associações.

pondem a identificadores naturais e que existem no domínio do negócio não devam ser definidos. Exemplos desses atributos são `CPF`, `código` (de um produto), `númeroMatrícula` (de um aluno) etc.

Ainda sobre atributos, deve-se notar também que a sua definição para uma classe é dependente do contexto. Uma mesma classe, por exemplo, `Pessoa`, pode ter atributos diferentes em sistemas diferentes. Para um sistema que mantém informações sobre pessoas suspeitas de terem cometido crimes, os atributos dessa classe poderiam ser: fotografia, altura, peso, álibi, registro policial etc. Já em um sistema de recursos humanos que mantém informações sobre os empregados de uma companhia, os atributos poderiam ser: matrícula, salário, data de contratação etc.

5.5.2 Definição de associações

Na Seção 5.4.3, vimos que uma responsabilidade de conhecer pode ser mapeada para uma associação. Por outro lado, o fato de uma classe possuir colaboradores indica que devem existir relacionamentos entre estes últimos e a primeira. Isso porque um objeto precisa conhecer o outro para poder lhe fazer requisições. Portanto, para criar associações, verifique os colaboradores de uma classe, conforme identificados pela técnica de modelagem CRC. O raciocínio para definir associações reflexivas, ternárias e agregações é o mesmo, com a ressalva de que, em uma agregação, a classe em questão é o todo e o seu colaborador corresponde à parte.

Classes de associação (ver Seção 5.2.2.4) surgem a partir de responsabilidades de saber (e, mais raramente, de responsabilidades *de fazer*) que o modelador não conseguiu atribuir a alguma classe exclusivamente. Nessas situações, a solução é criar uma classe associativa para que ela cumpra tal responsabilidade. Por exemplo, considere a situação em que pessoas trabalham em projetos, formando uma associação de conectividade "muitos para muitos". Se houver a necessidade de conhecer quantas horas semanais cada pessoa trabalha em cada um dos projetos, essa responsabilidade não pode ser atribuída à classe Pessoa exclusivamente, pois isso violaria as propriedades 2 e 3 de um atributo, defini-

das na Seção 5.5.1 (é possível haver pessoas que não estejam alocadas em projetos; além disso, uma pessoa pode ter diversas cargas horárias diferentes, uma para cada projeto no qual trabalhe). Um raciocínio análogo se aplica à atribuição da responsabilidade em questão à classe Projeto, que também não é adequada para cumprir com a mesma. Portanto, há uma responsabilidade *de conhecer* que não se aplica exclusivamente a nenhuma das classes envolvidas. A solução é criar uma classe associativa para realizar essa responsabilidade de conhecer. A Figura 5-45 ilustra essa situação.

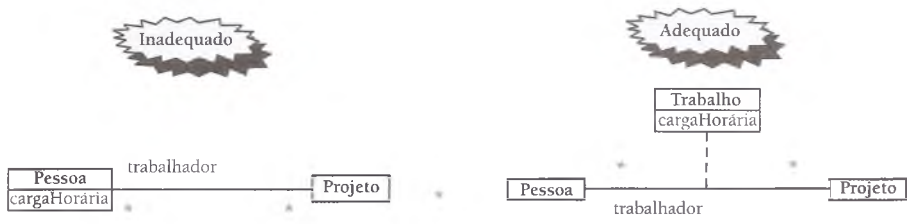


Figura 5-45: Definindo classes associativas para cumprir responsabilidades órfãs.

5.5.3 Organização da documentação

Uma vez que classes e suas responsabilidades e colaboradores tenham sido identificados, esses elementos devem ser organizados em um ou mais diagramas de classes e documentados. O conjunto de todos os diagramas de classes identificadas na análise de um sistema, juntamente com sua documentação (e eventuais diagramas de objetos; ver Seção 5.3), corresponde ao *modelo de classes de análise*.

Na diagramação, sempre que possível, as classes devem ser posicionadas de tal forma que as associações sejam lidas da esquerda para a direita ou de baixo para cima. Isso facilita a leitura, principalmente se o diagrama tiver de ser entendido por pessoas de cultura ocidental (onde se costuma ler da esquerda para a direita e de cima para baixo).

A construção de um único diagrama de classes para todo o sistema pode resultar em um diagrama bastante complexo. Uma alternativa é criar uma *visão de classes participantes* (VCP) para cada caso de uso. Uma VCP é um diagrama de classes cujas instâncias (objetos) participam de um caso de uso. Se essa estratégia for adotada, a elaboração de cada caso de uso resulta em uma VCP. A Seção 5.7 ilustra um exemplo de uma VCP construída para um caso de uso.

As visões de classes participantes de cada caso de uso podem ser reunidas para formar um único diagrama de classes para o sistema como um todo. Se esse diagrama ficar muito grande, o modelador pode optar por "esconder" as classes de fronteira ou até mesmo as classes de controle. Uma ferramenta CASE que dê suporte a essa operação seria de grande ajuda para a equipe de desenvolvimen-

to. Além disso, o recurso de modularização representado pelo mecanismo de *pacotes* (ver Seção 3.5) é também útil. Descrevemos a construção de diagramas de pacotes para classes no Capítulo 11.

Além do diagrama, as classes devem ser documentadas textualmente. Inicialmente, pode-se utilizar o próprio formato dos cartões CRC como documentação (considerando que esta técnica de identificação é utilizada). Adicionalmente, uma definição (de uma ou duas frases) deve ser fornecida para cada classe, assim como para cada um de seus atributos cujo significado não seja fácil de inferir pelo nome. À medida que o desenvolvimento evolui, é adequado criar um *glossário* para definir os termos utilizados na construção dos modelos (ver Seção 5.7.2).

Os nomes escolhidos para as classes devem estar no singular (pois uma classe já representa vários objetos), iniciando com letra maiúscula. As classes devem preferencialmente receber nomes provenientes do domínio, ou inspirados neste. As responsabilidades atribuídas a cada classe (que resultam em atributos e operações) também devem receber nomes provenientes ou inspirados no domínio.

Por fim, é também útil adicionar uma seção de detalhes de implementação ao modelo de classes, assim como para o modelo de casos de uso (ver Seção 4.4.3.14). Embora os detalhes de implementação não façam parte nem devam ser considerados no modelo de classes de análise, esses detalhes inevitavelmente surgem na mente dos analistas enquanto estão construindo esse modelo. Esta seção de implementação serve como repositório de idéias e soluções relativas ao modelo de classes que podem ser utilizadas (ou mesmo descartadas) nas atividades futuras do desenvolvimento.

5.6 Modelo de classes no processo de desenvolvimento

Em um desenvolvimento *dirigido a casos de uso* (ver Seção 4.6), após a descrição dos casos de uso essenciais, é possível realizar a identificação de classes. Nesse ponto, uma ou mais das técnicas de identificação de classes (descritas na Seção 5.4) podem ser aplicadas. Durante a aplicação dessas técnicas, as classes identificadas são refinadas para retirar inconsistências e redundâncias. Finalmente, as classes são documentadas e o diagrama de classes inicial é construído, resultando no modelo de classes de análise.

Depois que a primeira versão do modelo de classes de análise está completa, o modelador deve retornar ao modelo de casos de uso e verificar a consistência entre os dois modelos. Uma inconsistência indica a existência de algum erro no modelo de casos de uso ou no modelo de classes. Um caso de uso em que não foram identificados objetos, ou uma classe que não participa da realização de algum caso de uso são exemplos de inconsistências que devem ser eliminadas.

Pelo exposto anteriormente, pode-se notar que as construções do modelo de casos de uso e do modelo de classes são *retroativas* entre si. Por exemplo, na realização de uma sessão CRC, novos casos de uso podem ser identificados, ou pode-se identificar a necessidade de modificação de casos de uso preexistentes. A Figura 5-46 ilustra a interdependência entre a construção do modelo de casos de uso e o modelo de classes. De fato, esta é mais uma conseqüência do modo de pensar orientado a objetos: detalhes são adicionados aos poucos, à medida que o problema é entendido.

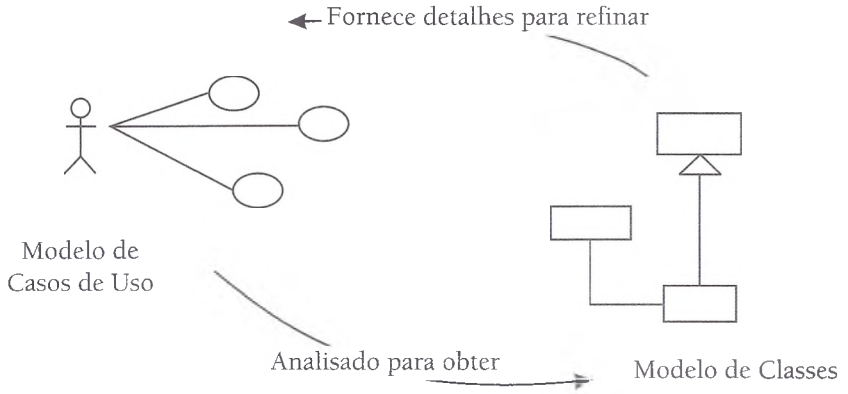


Figura 5-46: Interdependência entre o modelo de casos de uso e o modelo de classes.

Na Seção 2.3.2, descrevemos o ciclo de vida de desenvolvimento iterativo. Nesse ciclo de vida, os artefatos de software são construídos aos poucos, iteração após iteração. Além dessa característica de construção incremental, os diversos modelos do SSOO, quando construídos, fornecem informações úteis para refinar os modelos preexistentes. Em relação ao modelo de classes de análise, a coisa não acontece diferente. Na construção deste, a ênfase inicial recai em identificar os objetos do domínio. Após isso, alguns relacionamentos, atributos e mesmo operações desses objetos podem ser identificados nesta fase. Em seguida, alguns objetos da aplicação (particularmente objetos de fronteira e de controle) são também identificados na análise. Entretanto qualquer elemento identificado na modelagem de classes de análise está sujeito a modificações e refinamentos, em função das informações obtidas com a *modelagem dinâmica*. Primeiramente, a modelagem dinâmica fornece detalhes adicionais que podem ser utilizados para refinar os elementos já identificados do modelo de classes. Em segundo lugar, pode ser que alguma classe ou responsabilidade importante para o sistema só venha a ser descoberta quando o aspecto dinâmico do sistema for considerado.

O aspecto dinâmico de um SSOO é representado pelo *modelo de interações* e pelo *modelo de estados*. A construção desses modelos é possível através de infor-

mações obtidas com a construção do modelo de classes. Por outro lado, com a construção desses modelos, o modelador obtém informações suficientes para retornar ao modelo de classes e refiná-lo.

Assim como a modelagem estática (modelagem de classes), a modelagem dinâmica também começa na fase de análise. A modelagem dinâmica é representada pelo *modelo de interações* e pelo *modelo de estados*. O modelo de interações é objeto de estudo do Capítulo 7. A descrição do modelo de estados está no Capítulo 10.

Para finalizar esta seção, um comentário de viés um tanto filosófico. Podemos perceber que não só um SSOO é composto de entidades que colaboram entre si. A própria modelagem orientada a objetos é também constituída de entidades (modelos), cada uma "colabora" com informações para a construção da outra. Sendo assim, o aspecto dinâmico depende do aspecto estrutural estático, e vice-versa. Nenhum é mais importante que o outro; um serve para completar o outro.

5.7 Estudo de caso

Esta seção continua o desenvolvimento da modelagem do estudo de caso iniciado na Seção 4.7. Aqui, é apresentado o modelo de classes de domínio inicial desse estudo de caso. Conforme descrito neste capítulo, o processo de construção do modelo de classes de domínio é realizado por meio da análise dos casos de uso. Para cada um deles, os modeladores identificaram quais classes são necessárias para que os resultados externamente visíveis sejam obtidos.

Tome como exemplo o caso de uso denominado Realizar Inscrição. Conforme mencionado na Seção 5.4.2.5, cada caso de uso tem, a princípio, um objeto de fronteira para cada ator e um objeto controlador. Com base nessa heurística, as primeiras classes identificadas para o cenário principal desse caso de uso foram `ControladorInscrição`, `FormulárioInscrição` e `SistemaFaturamento`. Essas duas últimas são classes de fronteira do sistema que realizam a interface com os atores `Aluno` e `Sistema de Faturamento`, respectivamente. Note que essas classes de fronteira têm naturezas completamente diferentes. Uma corresponde possivelmente a uma interface gráfica com o usuário. A outra corresponde à implementação de algum protocolo de comunicação. Entretanto, na análise, não devemos nos ater a esses detalhes. O importante é identificarmos que tais classes são necessárias para prover as funcionalidades (requisitos funcionais) do sistema.

A identificação das classes de entidade envolvidas na realização de certo caso de uso se dá mediante a aplicação de uma ou mais das técnicas descritas na Seção 5.4. Por exemplo, quando utilizamos a técnica de análise de casos de uso (ver Seção 5.4.2), estudamos a descrição de cada passo do fluxo principal do caso de uso em que o sistema realiza alguma ação. Cada um desses passos impli-

ca classes e responsabilidades dentro do sistema. Durante a identificação de classes a partir do texto de um caso de uso, os fluxos alternativos e de exceção devem também ser analisados. Por exemplo, através da análise do fluxo alternativo Inclusão em lista de espera (ver Seção 4.7.3), identificamos a necessidade da classe `ListaEspera`, responsável por manter uma lista de alunos que estão esperando pela abertura de mais uma oferta para uma determinada disciplina (ver Figura 5-47). Note que essa classe nem mesmo é mencionada no fluxo principal do caso de uso, o que enfatiza a necessidade de avaliarmos todos os fluxos de um caso de uso durante a identificação.

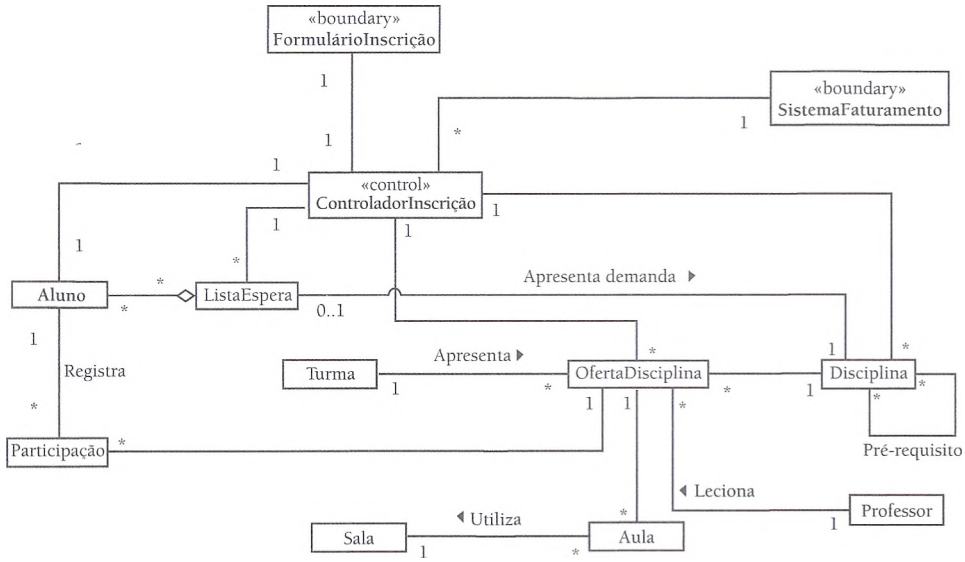


Figura 5-47: VCP para o caso de uso Realizar Inscrição.

A visão de classes participantes para o caso de uso `Realizar Inscrição` é apresentada na Figura 5-47. Alguns diagramas correspondentes às visões de classes participantes de outros casos de uso do sistema são apresentados logo a seguir. Como exercício, o leitor é convidado a elaborar as visões de classes participantes que não são apresentadas aqui, com base nos casos de uso descritos na Seção 4.7.3.

Note que os diagramas aqui apresentados não exibem atributos nem operações das classes. Embora o mapeamento de determinadas propriedades das classes já possa ser feito neste momento, essa tarefa é adiada para os capítulos seguintes, quando descrevemos o modelo de interações (Capítulo 7) e o modelo de classes de projeto (Capítulo 8). Isso porque é graças à construção do modelo de interações de um sistema que identificamos as operações que uma classe deve possuir.

Outra característica geral que podemos notar em cada VCP aqui apresentada é que o objeto de fronteira associado ao ator primário do caso de uso normalmente contém o controlador do caso de uso.

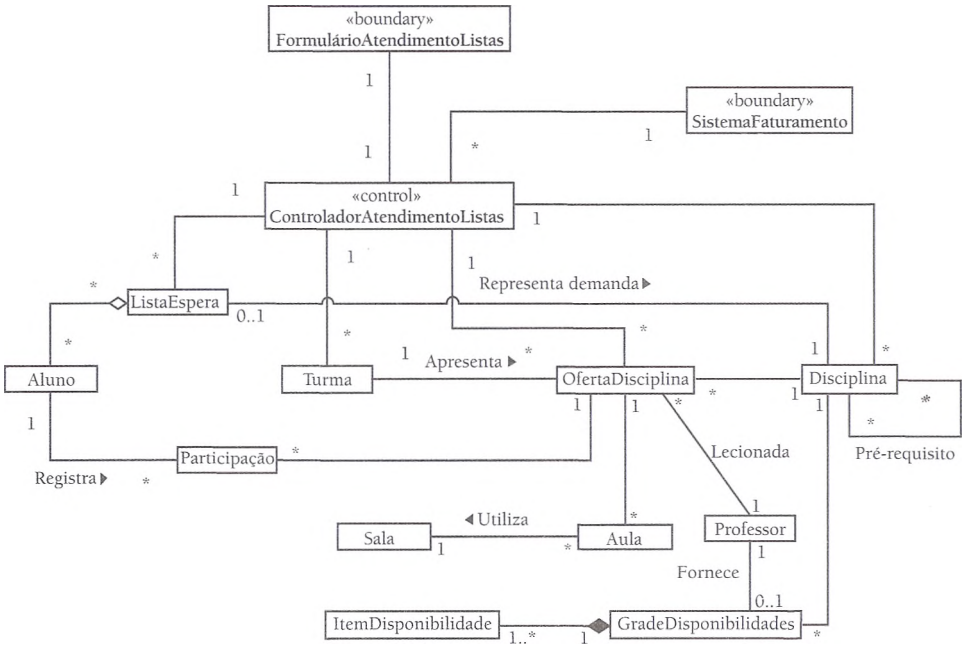


Figura 5-48: VCP para o caso de uso Atender Listas de Espera.

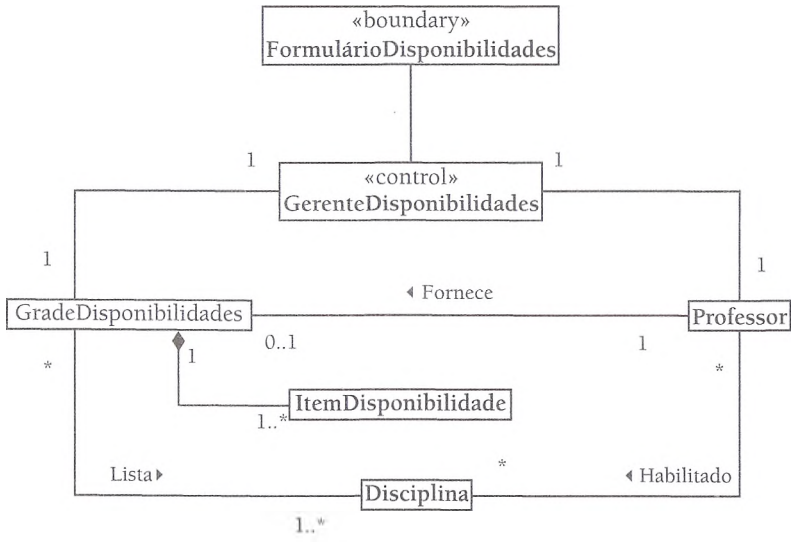


Figura 5-49: VCP para o caso de uso Fornecer Disponibilidades.

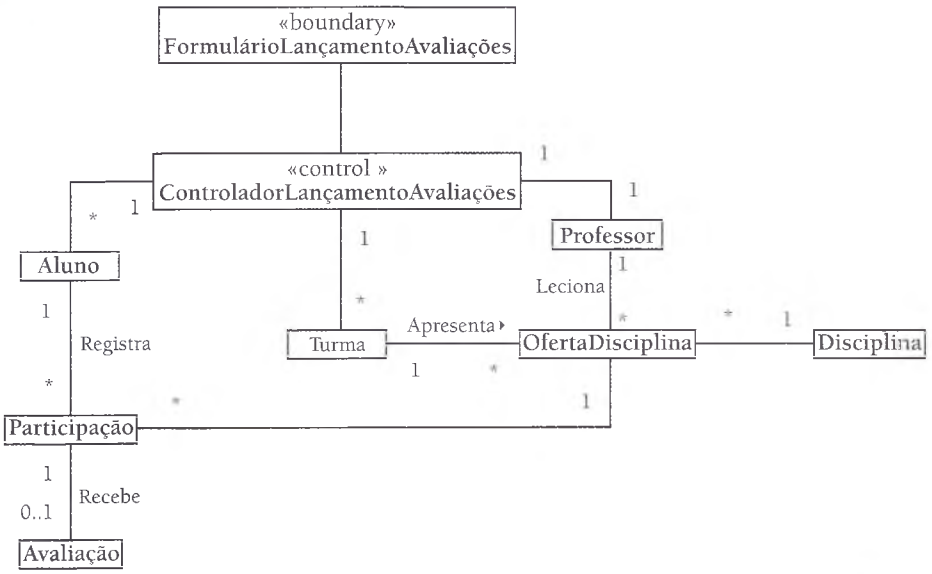


Figura 5-50: Visão de classes participantes para o caso de uso Lançar Avaliações e Freqüências.

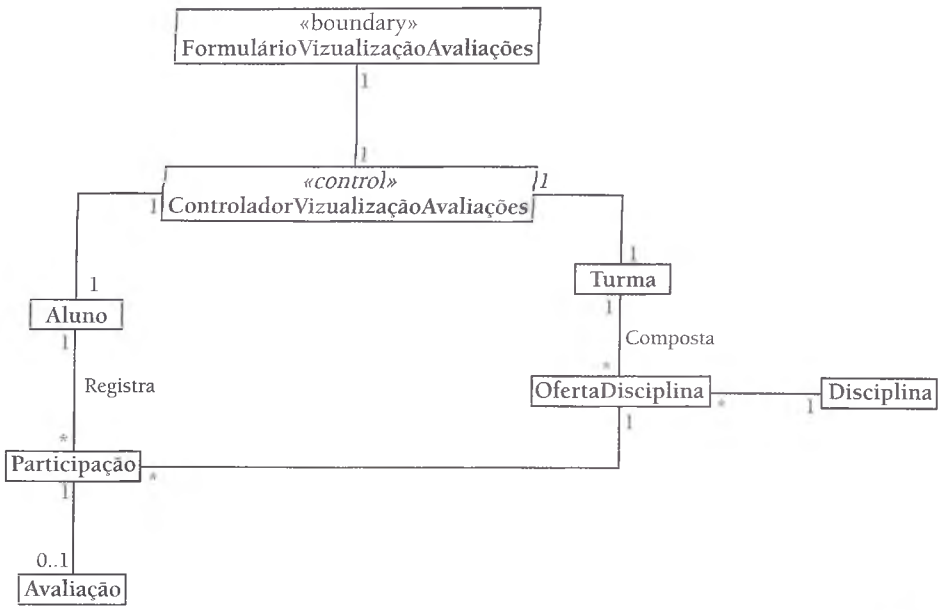


Figura 5-51: Visão de classes participantes para o caso de uso Visualizar Avaliações e Freqüências.

Note que nos diagramas de classes representando visões de classes participantes apresentados aqui, os nomes das associações entre controladores e classes de entidade e entre controladores e classes de fronteira não são apresentados. Isso

se justifica pelo fato de uma associação entre objetos controladores e demais objetos de um sistema não ser duradoura (só existe em tempo de realização do caso de uso) e normalmente tem uma mesma semântica básica: a de que esses demais objetos são coordenados pelo controlador para realizar alguma tarefa. Ainda sobre essas associações, note que as apresentamos aqui no início de nosso estudo de caso. Entretanto, a identificação dessas associações é realizada mediante a construção do modelo de interações, que estudamos no Capítulo 7.

A Figura 5-52 apresenta um diagrama de classes em que somente são apresentadas as classes do domínio do problema, ou seja, as classes de entidade. Esse diagrama corresponde a uma *unificação* das visões de classes participantes apresentadas anteriormente. Isso porque, conforme podemos perceber pelas VCPs, diversas classes de entidades aparecem em mais de uma VCP. Entretanto, note que esse diagrama é somente a versão inicial do modelo de classes do SCA. Durante a fase de projeto, diversas outras classes surgem. Não que outro modelo de classes deva ser construído; em vez disso, o que acontece é que o modelo de classes de análise é estendido para ser transformado no modelo de classes de projeto.

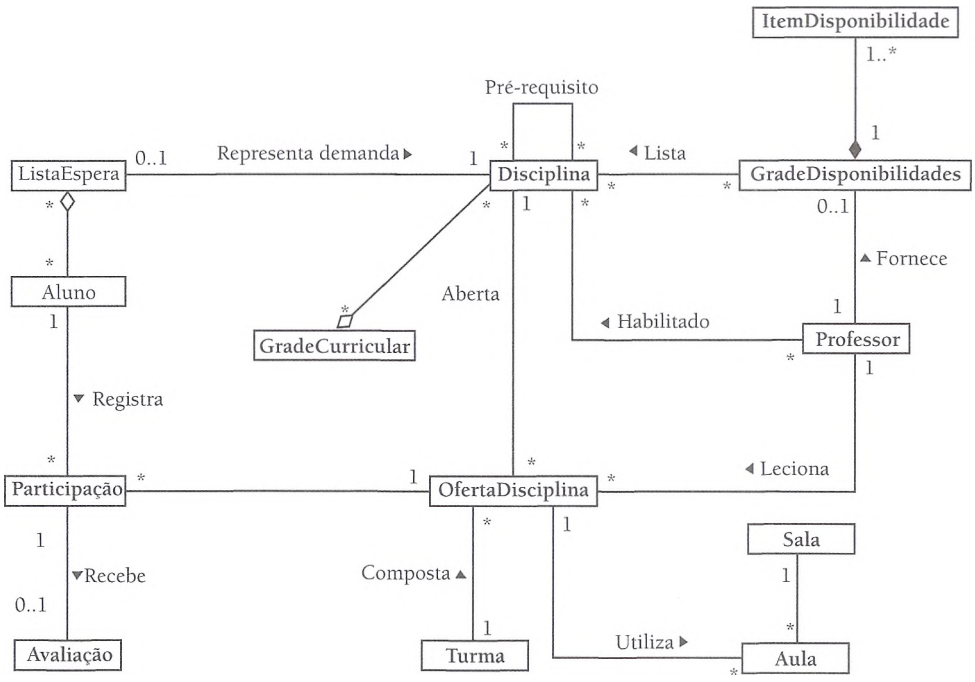


Figura 5-52: Diagrama de classes que apresenta as classes de entidade do SCA.

5.7.1 Cartões CRC

Conforme vimos na Seção 5.4.3.1, outra técnica para identificação de classes de um sistema é a modelagem CRC. Vimos também que a aplicação de uma técnica não exclui a outra. Portanto, nesta seção, apresentamos cartões CRC para os principais cenários dos casos de uso do SCA. Com a construção desses cartões, chegamos a um conjunto de *responsabilidades* para cada classe do sistema. Os cartões construídos durante as sessões CRC são apresentados a seguir.

Disciplina	
Responsabilidades	**Colaboradores**
1. Conhecer seus pré-requisitos	Disciplina
2. Conhecer seu código	
3. Conhecer seu nome	
4. Conhecer sua quantidade de créditos	

ListaEspera	
Responsabilidades	**Colaboradores**
1. Conhecer a sua disciplina	Disciplina
2. Manter os alunos em espera pela abertura de vagas	Aluno

Aluno	
Responsabilidades	**Colaboradores**
1. Conhecer seu número de registro	Participação
2. Conhecer seu nome	
3. Conhecer as disciplinas que já cursou	

Participação	
Responsabilidades	**Colaboradores**
1. Conhecer sua oferta de disciplina	OfertaDisciplina
3. Conhecer sua avaliação	Avaliação

Turma	
Responsabilidades	**Colaboradores**
1. Conhecer o seu código	OfertaDisciplina
2. Conhecer as disciplinas que oferece	

OfertaDisciplina	
Responsabilidades	Colaboradores
1. Conhecer a sua situação	Disciplina
2. Conhecer sua turma	Turma
3. Conhecer a sua disciplina	Professor
4. Conhecer o ano e o semestre letivo em que acontece	Aula
5. Conhecer dias, horários e salas de aula em que acontece	
6. Conhecer sua quantidade máxima de alunos	
7. Conhecer seu professor	

ControladorInscrição	
Responsabilidades	Colaboradores
1. Conhecer as disciplinas de um semestre letivo	Aluno
2. Procurar uma turma disponível para inscrever um aluno em uma disciplina	Disciplina
3. Conhecer as ofertas para uma disciplina	OfertaDisciplina
4. Verificar a possibilidade de inscrição de aluno em uma disciplina	ListaEspera
5. Informar ao aluno os detalhes de sua inscrição em uma disciplina	SistemaFaturamento
6. Inserir um aluno na lista de espera de uma disciplina	

FormulárioInscrição	
Responsabilidades	Colaboradores
1. Receber requisições de inscrição de um aluno	ControladorInscrição
2. Exibir uma lista de disciplinas nas quais um aluno pode se inscrever	
3. Exibir os resultados de inscrição de um aluno	

Todas as responsabilidades identificadas durante a modelagem CRC devem ser alvo de validação durante a *modelagem de interações* do sistema. No Capítulo 7, estudamos essa atividade tão importante da modelagem de um SSOO e descrevemos sua correlação com as responsabilidades identificadas pela modelagem CRC.

5.7.2 Glossário

Os termos relevantes do domínio, a maioria deles correspondentes a classes identificadas, foram definidos no glossário do SCA. A definição inicial desse glossário é apresentada a seguir:

1. **Aluno**: representa um aluno da faculdade.
2. **Aula**: representa o acontecimento semanal de uma aula de alguma oferta de disciplina. Toda aula acontece em uma sala da faculdade.
3. **Avaliação**: representa uma avaliação atribuída à participação de um aluno em uma turma que oferece alguma disciplina da faculdade.
4. **Disciplina**: uma disciplina componente da grade curricular da faculdade.
5. **Grade de Disponibilidades**: representa a grade de dias da semana e os respectivos horários no semestre letivo seguinte ao atual nos quais um professor está disponível para lecionar na faculdade. Representa também as disciplinas que o professor está apto a lecionar nesse mesmo semestre letivo.
6. **Grade Curricular**: uma grade curricular corresponde a um conjunto de disciplinas, juntamente com os pré-requistos de cada uma delas. De tempos em tempos, a instituição de ensino atualiza a sua grade curricular. Nessa atualização, novas disciplinas podem ser criadas, assim como disciplinas existentes podem ser removidas da grade.
7. **Lista de Espera**: representa uma lista dos alunos que estão esperando que uma certa disciplina seja oferecida em alguma turma.
8. **Oferta de disciplina**: representa a alocação de uma disciplina em alguma turma.
9. **Participação**: representa a participação de um aluno em alguma disciplina ofertada pela faculdade. A participação de um aluno em uma oferta de disciplina possui uma avaliação.
10. **Professor**: representa o indivíduo que leciona as disciplinas ofertadas pela faculdade.
11. **Sala**: representa um dos locais da faculdade que podem ser utilizados para as aulas de uma ou mais disciplinas ofertadas pela faculdade.
12. **Turma**: representa uma turma aberta em algum semestre letivo da faculdade. Uma turma possui diversas disciplinas ofertadas.
13. **Diário de Classe**: corresponde a um formulário que contém os nomes dos alunos inscritos em determinada oferta de disciplina. Nesse formulário, o professor lança a quantidade de faltas e as avaliações de cada aluno.

EXERCÍCIOS

5-1: Descreva a posição do diagrama de classes no processo de desenvolvimento incremental e iterativo. Quando eles são utilizados? Para que são utilizados?

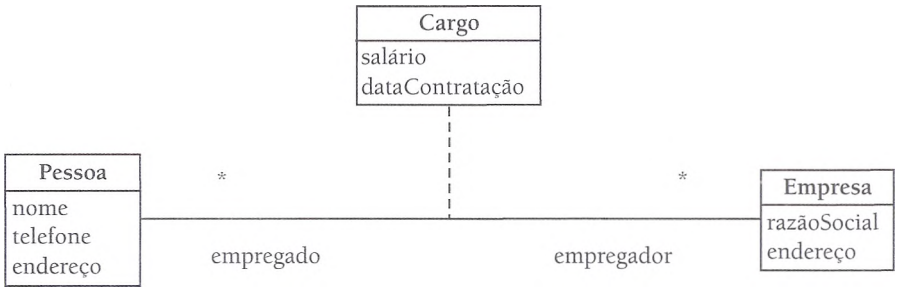
5-2: Considere a técnica CRC. Discuta a relação existente entre as dimensões espaciais usuais de um cartão CRC e a distribuição quase uniforme das responsabilidades.

5-3: Construa o modelo de classes de domínio de um sistema de informações para controlar o campeonato da Fórmula 1.

5-4: Desenhe um diagrama de classes com relacionamentos, nomes de papéis e multiplicidades para as seguintes situações:

Uma Pessoa pode ser casada com outra Pessoa.
Uma Disciplina é pré-requisito para outra Disciplina.
Uma Peça pode ser composta de diversas outras Peças.

5-5: Considere o diagrama de classes a seguir, que exibe uma classe associativa entre as classes Pessoa e Empresa. Crie um diagrama de classes equivalente ao fornecido a seguir, mas sem utilizar uma classe associativa.



5-6: Construa um diagrama de classes inicial para a seguinte situação: *Pacotes são enviados de uma localidade a outra. Pacotes têm um peso específico. Localidades são caracterizadas pelas facilidades de transporte (por exemplo, rodoviárias, aeroportos e auto-estradas). Algumas localidades são vizinhas, isto é, existe uma rota direta de transporte entre tais localidades. A rota de transporte entre as localidades tem um certo comprimento (a distância entre as localidades). Trens, aviões e caminhões são usados para o transporte de pacotes. Cada um destes meios de transporte pode suportar uma carga máxima de peso. A cada momento, durante o seu transporte, é necessário saber a posição (localidade) de cada pacote. Também é necessário manter o controle de que meio de transporte está sendo utilizado em cada parte da rota para um certo pacote.*

5-7: Considere o seguinte discurso relativo a um sistema de partidas de tênis: "Num torneio de tênis, cada partida é jogada entre dois jogadores. Pretende-se manter informação sobre o nome e a idade dos jogadores; data da partida e atribuição dos jogadores às partidas. O máximo de partidas que um jogador poderá realizar são seis e o mínimo uma". Desenhe o diagrama de classes correspondente.

5-8: Desenhe um diagrama equivalente ao da Figura 5-10 de duas formas:
a) Utilizando uma classe ordinária para substituir a classe associativa.
b) Utilizando uma associação ternária.

5-9: Identifique classes e/ou relacionamentos a partir das seguintes regras do negócio:
a) Pedidos são compostos de vários itens de pedido.
b) Um item de pedido diz respeito a um e exatamente um produto.
c) Um pedido pode conter até 20 itens.

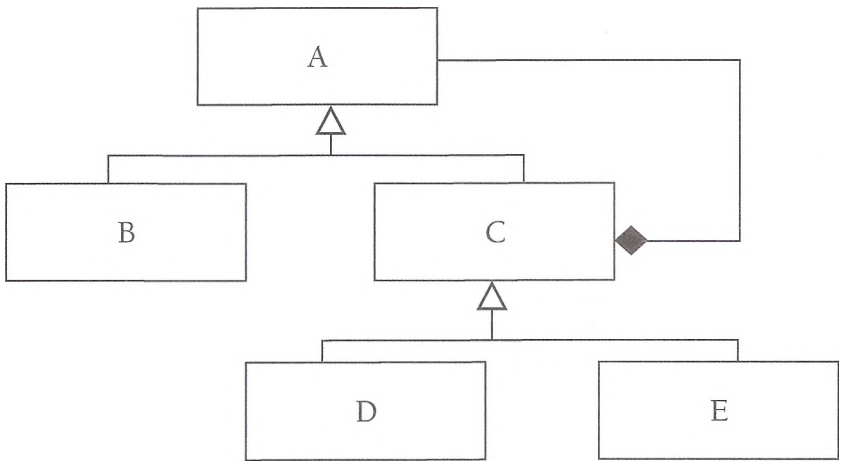
5-10: Considere um sistema de software para controlar um hotel. Normalmente, um *hóspede* ocupa um *quarto* por *estada*. Mas, suponha que uma nova regra foi criada no negócio: agora, um hóspede pode utilizar até três quartos. Desenhe o diagrama de classe para essas duas situações.

5-11: Reflita sobre a seguinte afirmação: "O tamanho do cartão CRC ajuda a limitar e a restringir a complexidade das classes identificadas nas sessões CRC."

5-12: Reflita e discuta com algum colega sobre a seguinte afirmação: "*Atributos são similares a associações. Um atributo de uma classe é apenas uma notação para associá-la a um conceito que tem um valor atômico*."

5-13: A seguir, são enumeradas diversas responsabilidades típicas de serem encontradas em objetos de um sistema de software. Discuta qual das categorias de objetos (fronteira, controle ou entidade) é mais adequada para cumprir cada uma dessas responsabilidades:
a) Criação ou destruição de um objeto.
b) Formação ou destruição de associações entre objetos de entidade.
c) Obtenção ou modificação de valores de atributos de um objeto de entidade.
d) Exibição de mensagens para o ator.
e) Realização de cálculos complexos.

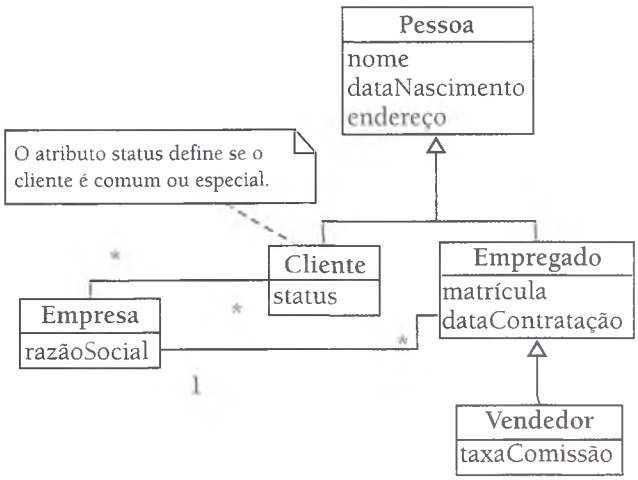
5-14: Considere as instâncias das classes do diagrama exibido abaixo.



Para o diagrama de classes apresentado, qual das seguintes situações são possíveis?

1. e1 contém um d1, o qual contém um e2, o qual contém um b2.
2. a1 contém um c1, o qual contém um d1.
3. b1 contém um d1, o qual contém um e2 .
4. c1 contém um a1, o qual contém b1.

5-15: Considere o diagrama de classes a seguir. Há como construir um modelo equivalente, sem o uso de gen/spec? Qual seria a vantagem dessa transformação?



5-16: Analise os dois fragmentos de diagrama de classes a seguir. Eles são equivalentes? Explique sua resposta.